

TERRAFORM SECTION

• STARTS HERE •

Topic : Terraform

Date : 01/08/2026

@codewithanush

Infrastructure as Code

Multi-Cloud
Ready



Modular
& Reusable



Declarative
Language (HCL)



Safe
& Reliable

TERRAFORM

Build • Change • Manage
Your Infrastructure

What You Will Learn



Write Infrastructure
as Code



Manage Resources
Across Clouds



Automate, Version
& Collaborate



Secure, Scalable
& Cost Effective



Consistent
& Repeatable

Terraform Workflow



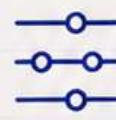
Write
(Code)



Plan
(Preview)



Apply
(Provision)



Manage
(Operate)



Destroy
(Clean Up)

TERRAFORM

T1. INTRODUCTION TO TERRAFORM & INFRASTRUCTURE AS CODE

Topic : Terraform

Date : 01/08/2026

@ codewithanush

1. What is Terraform ?

Terraform is an **open-source** Infrastructure as Code (IaC) tool created by **HashiCorp**. It allows you to define, provision and manage infrastructure across multiple cloud providers in a safe, efficient and repeatable way.



HashiCorp
Terraform

Key Highlights

★ Infrastructure as Code (IaC)



★ Multi-Cloud Support



★ Declarative Language (HCL)



★ Plan, Preview & Apply Changes



★ State Management



★ Reusable & Modular



★ Automation & Consistency



2. What is Infrastructure as Code (IaC) ?

Infrastructure as Code (IaC) means managing and provisioning infrastructure using **code** instead of manual processes.

You write configuration files that describe the infrastructure, and Terraform takes care of creating and managing it.



Code
(HCL)



Infrastructure

3. Why Use Terraform ?

- ✓ Automate infrastructure provisioning
- ✓ Reduce manual errors
- ✓ Ensure consistency across environments
- ✓ Version control & collaboration
- ✓ Reusable & modular configurations
- ✓ Faster deployment & easy scaling
- ✓ Supports multiple cloud providers

4. How Terraform Works (High Level)



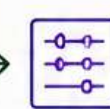
Write
(Code)



Plan
(Preview)



Apply
(Provision)



Manage
(Operate)



Destroy
(Clean up)



Quick Revision

- Terraform helps you build, change and manage infrastructure using code.
- It makes infrastructure provisioning consistent, repeatable and reliable.
- Write once, use anywhere!

Code Your
Infrastructure,
Simplify Operations!

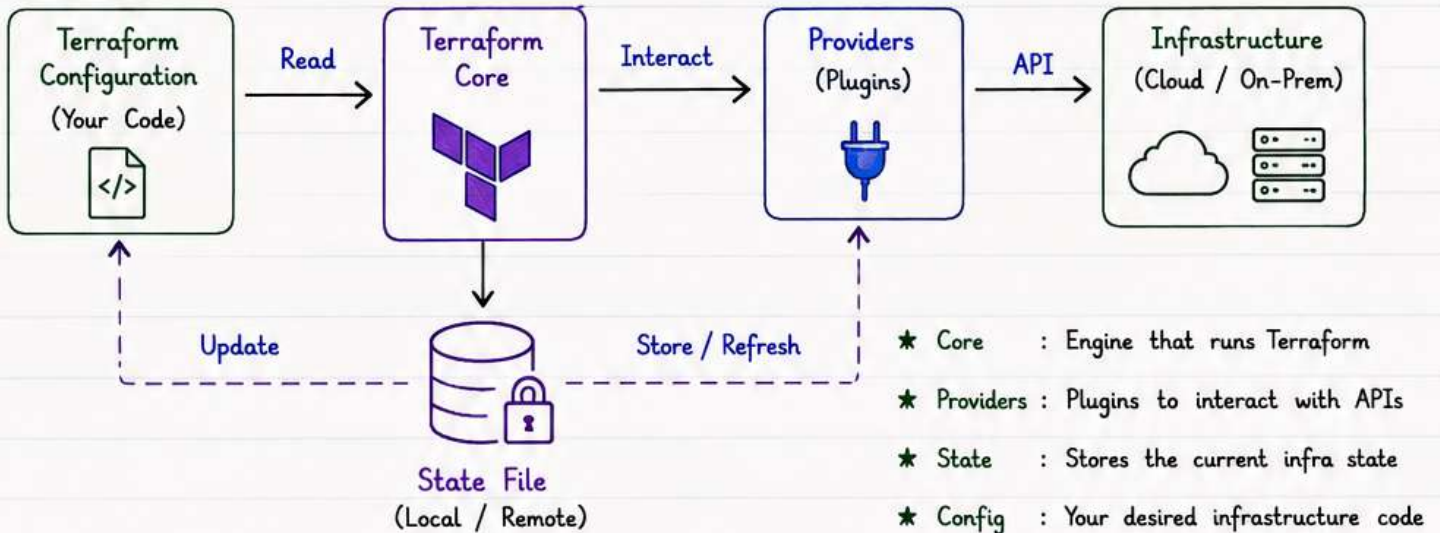


@ codewithanush

T2. TERRAFORM ARCHITECTURE & WORKFLOW

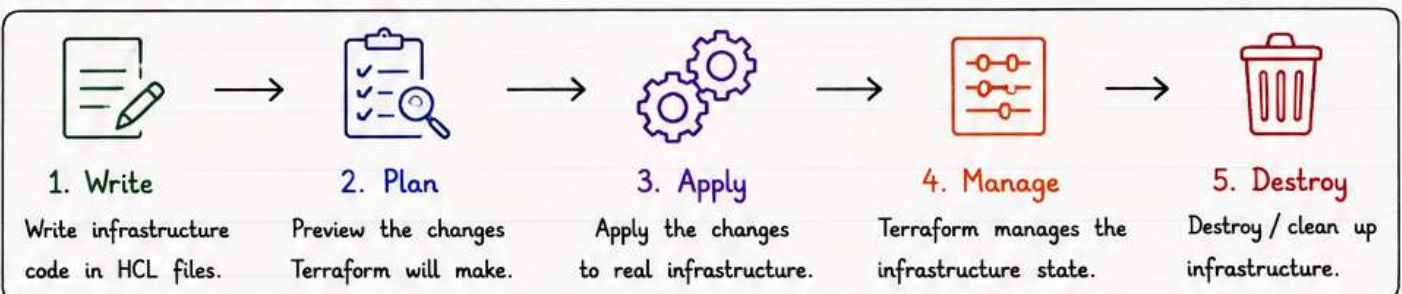
1. Terraform Architecture

Terraform follows a simple **client-server** architecture.



2. Terraform Workflow (High Level)

Terraform follows a **declarative** workflow.



3. Key Components

- ★ Configuration Files (.tf) : Define desired state
- ★ Providers : Connect to APIs (AWS, Azure, GCP, etc.)
- ★ Resources : Infrastructure components (EC2, VPC, S3, etc.)
- ★ State File : Tracks current state of infrastructure
- ★ Variables : Make configurations dynamic & reusable
- ★ Outputs : Show important information after apply

4. State File Role

- ★ Acts as Terraform's source of truth.
- ★ Maps real infrastructure to your configuration.
- ★ Stored locally or remotely (for team collaboration).
- ★ Locking prevents concurrent changes & corruption.



Quick Tip

- Always run `terraform plan` before `terraform apply`.
- State file is very important. **Protect** it and never edit manually.
- Use **remote state** & locking in team environments.

Code Your
Infrastructure,
Automate Everything!

T3. TERRAFORM INSTALLATION & SETUP

1. Prerequisites

- ★ Operating System : Windows / macOS / Linux
- ★ Internet Connection
- ★ Admin / `sudo` access (for installation)
- ★ A Text Editor or IDE (VS Code recommended)

2. Install Terraform

Windows	macOS	Linux (Ubuntu/Debian)
- Download Terraform zip from https://developer.hashicorp.com/terraform/downloads - Extract the zip file - Move terraform.exe to a folder (e.g. <code>C:\terraform</code>) - Add the folder to System Environment PATH 	- Using Homebrew (Recommended) <code>brew install terraform</code> - OR - Download the .zip from official website - Unzip and move to a directory (e.g. <code>/usr/local/bin</code>) 	- Download the official package <code>wget https://releases.hashicorp.com/terraform/latest/terraform-linux_amd64.zip</code> - Unzip the file <code>unzip terraform_*.zip</code> - Move binary to <code>/usr/local/bin</code> <code>sudo mv terraform /usr/local/bin/</code> - Give execute permission <code>sudo chmod +x /usr/local/bin/terraform</code> 
☆ Now Terraform is installed on your system.		

3. Verify Installation

- ★ Check Terraform version

```
terraform -version
```

You should see output like this :

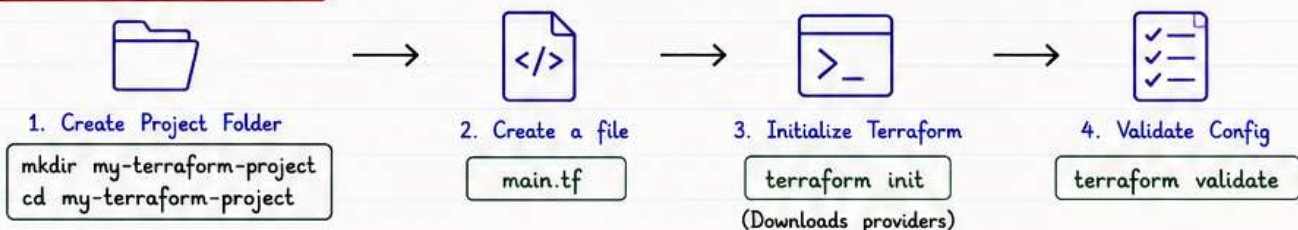
```
Terraform v1.7.x
on windows_amd64 (or linux_amd64)
+ provider registry.terraform.io/hashicorp/aws v5.x.x
Your version of Terraform is up to date!
```

4. Setup IDE (VS Code Recommended)

- ★ Install VS Code
- ★ Install HashiCorp Terraform Extension
- ★ Useful extensions:
 - HashiCorp Terraform
 - YAML
- ★ Create a new folder for your Terraform project.



5. First Project Setup



Quick Tip

- Always use latest stable version of Terraform.
- Keep Terraform binary in PATH for easy access.
- Use `terraform init` in every new project.

Code Your Infrastructure,
Automate Everything!

T4. TERRAFORM CLI - IMPORTANT COMMANDS

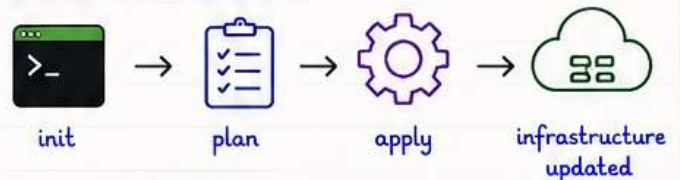
Terraform **CLI** (Command Line Interface) is used to initialize, plan, apply and manage your infrastructure as code.



No.	Command	Description	Example
①	terraform init	Initializes a working directory. Downloads provider plugins and initializes the backend.	terraform init
②	terraform validate	Validates the configuration files in the working directory.	terraform validate
③	terraform fmt	Rewrites config files to a consistent format.	terraform fmt
④	terraform plan	Creates an execution plan. Shows what changes Terraform will make.	terraform plan
⑤	terraform apply	Applies the changes required to reach the desired state.	terraform apply
⑥	terraform destroy	Destroys all managed infrastructure.	terraform destroy
⑦	terraform show	Shows the current state or a saved plan in human-readable form.	terraform show
⑧	terraform state list	Lists all resources in the state.	terraform state list
⑨	terraform state show <res>	Shows details of a specific resource in the state.	terraform state show aws_instance.web
⑩	terraform taint <res>	Marks a resource for recreation during next apply.	terraform taint aws_instance.web
⑪	terraform untaint <res>	Removes the taint from a resource.	terraform untaint aws_instance.web
⑫	terraform import <res> <id>	Imports an existing resource into Terraform state.	terraform import aws_s3_bucket.mybucket my-bucket
⑬	terraform refresh	Updates the state file with real infrastructure.	terraform refresh
⑭	terraform output	Shows output values from the root module.	terraform output
⑮	terraform workspace	Manages workspaces.	terraform workspace list
⑯	terraform version	Shows Terraform version.	terraform version

Quick Tips

- Always run 'terraform init' first.
- Always run 'terraform plan' before 'apply'.
- Review the plan carefully before applying changes.
- Use 'terraform fmt' to keep code clean.

Command Execution Flow

★ Most Used Commands : init → plan → apply → destroy

T5. HCL - TERRAFORM CONFIGURATION LANGUAGE

HCL (HashiCorp Configuration Language) is the declarative language used by Terraform to define infrastructure.

It is human readable and designed to describe configuration in blocks.



1. HCL Basics

- ★ HCL uses **blocks** to represent configuration.
- ★ Blocks have **labels** and **arguments**.
- ★ Arguments use key = value pairs.
- ★ HCL is case sensitive.
- ★ Comments start with **#** or **//**

2. Basic Structure of an HCL Block

```
<BLOCK_TYPE> "<LABEL1>" "<LABEL2>" {
  <ARGUMENT> = <VALUE>
  ...
}
```

→ Block Type

→ Labels (Optional)

→ Arguments

3. Example - Simple EC2 Instance Configuration

```
resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbf1afe1f0"
  instance_type = "t2.micro"
  key_name      = "my-key"
  tags = {
    Name = "MyEC2Instance"
    Environment = "Dev"
  }
}
```

- ★ **resource** : Block type (inbuilt Terraform block)
- ★ **aws_instance** : First label (Provider + Resource type)
- ★ **example** : Second label (Logical name)
- ★ **Arguments** : Key = Value pairs inside the block
- ★ **tags** : Map of key-value pairs

4. Common HCL Elements

Element	Example	Description
Block	<code>resource "aws_s3_bucket" "b" { ... }</code>	Defines a configuration block
Argument	<code>bucket = "my-bucket"</code>	Key = Value inside a block
String	<code>"hello"</code>	Text value
Number	<code>42</code>	Numeric value
Boolean	<code>true / false</code>	Boolean value
List	<code>["a", "b", "c"]</code>	Ordered collection
Map	<code>{ Name = "Anush" }</code>	Key-value collection

5. Comments in HCL

```
# This is a single line comment
// This is also a single line comment

/*
  This is a
  multi-line
  comment
*/
```

Comments are ignored by Terraform.

6. HCL File Extension

- ★ Terraform configuration files use **.tf** extension



→ **main.tf**
(Configuration File)

7. Key Points

- ★ HCL is declarative, not procedural.
- ★ You describe the **desired state**, Terraform handles the rest.
- ★ HCL is designed to be **simple, readable and maintainable**.



Quick Tip

- Always run 'terraform fmt' to format HCL code.
- Use 'terraform validate' to check syntax.
- Keep your HCL files clean and modular.

Write Code Once,
Provision Anywhere!

T6. PROVIDERS, RESOURCES & DATA SOURCES

Terraform uses **Providers** to communicate with cloud platforms.

You define **Resources** to create infrastructure and use **Data Sources** to fetch information about existing infrastructure.



AWS



Azure



GCP

1. Providers

- ★ A provider is a plugin that lets Terraform interact with a platform or service.
- ★ Examples: **aws**, **azurerm**, **google**, **random**, **kubernetes**, etc.
- ★ Configure provider with credentials & region.

Example - AWS Provider

```
provider "aws" {
  region      = "ap-south-1"
  access_key  = "AKIAxxxxxxxxxxxx"
  secret_key  = "xxxxxxxxxxxxxxxxxxxxxxxxxxxx"
}
```



Note:

Credentials can also be loaded from environment variables or IAM roles.

2. Resources

- ★ Resources are the infrastructure objects managed by Terraform.
- ★ You define the desired state of infrastructure using resource blocks.
- ★ Terraform will create, update or delete based on configuration.

Example - AWS EC2 Instance (Resource)

```
resource "aws_instance" "web" {
  ami          = "ami-0c55b159cbf9e1f0"
  instance_type = "t2.micro"
  key_name     = "my-key"
  tags = {
    Name = "MyEC2Instance"
    Environment = "Dev"
  }
}
```

★ Key Points

- ✓ Resource type: **aws_instance**
- ✓ Logical name: **web**
- ✓ Arguments inside the block define the settings.

3. Data Sources

- ★ Data sources are used to fetch information about **existing resources**.
- ★ They do not manage infrastructure.
- ★ Useful for referencing values that already exist (VPC, AMI, Subnet, etc.).

Example - Fetch Existing AMI (Data Source)

```
data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
  }

  owners = ["099720109477"] # Canonical
}
```

★ Key Points

- ✓ Used with data keyword.
- ✓ Helps to read information.
- ✓ Result can be used in resources.

4. Using Resources & Data Sources Together

Data Source



Fetch existing information

Resource



Create / Manage infrastructure

Output / Use



Use values in other resources or outputs

Example

```
data "aws_vpc" "default" {
  default = true
}

resource "aws_subnet" "public" {
  vpc_id      = data.aws_vpc.default.id # using data source
  cidr_block  = "10.0.1.0/24"
  availability_zone = "ap-south-1a"
}
```

5. Summary

Concept	Purpose	Type	Example
Provider	Connects Terraform to a platform	Configuration Block	<code>provider "aws" { ... }</code>
Resource	Create / Manage infrastructure	Resource Block	<code>resource "aws_instance" "web" { ... }</code>
Data Source	Read / Fetch existing information	Data Block	<code>data "aws_vpc" "default" { ... }</code>



Quick Tips

- Always configure provider first.
- Use resources to manage infrastructure.
- Use data sources to reference existing infrastructure.
- Providers, resources & data sources are the building blocks of Terraform.

Understand,
Configure,
Automate!



T7. VARIABLES, LOCALS & OUTPUTS

Terraform provides variables to make configurations dynamic, locals to store intermediate values and outputs to display important information.



Variables



Locals



Outputs

1. VARIABLES

- ★ Variables make your code dynamic and reusable.
- ★ Defined using the **variable** block.
- ★ Values are provided in .tfvars file or via CLI.
- ★ Used with **var.variable_name**

Example - variables.tf

```
variable "region" {
  description = "AWS region to deploy resources"
  type        = string
  default     = "ap-south-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}
```

Example - terraform.tfvars

```
region      = "ap-south-1"
instance_type = "t2.micro"
```

★ Key Points

- ★ Variables improve reusability.
- ★ Provide default values if possible.
- ★ Sensitive data can be marked as **sensitive = true**.

2. LOCALS

- ★ Locals store intermediate values.
- ★ Defined using the **locals** block.
- ★ Useful to avoid repetition in values or expressions.

Example - locals.tf

```
locals {
  name_prefix = "demo-${var.environment}"
  common_tags = {
    Project   = "TerraformDemo"
    Environment = var.environment
    Owner     = "DevOps"
  }
}
```

Example - Usage in resource

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = var.instance_type
  tags          = local.common_tags
}
```

★ Key Points

- ✓ Locals reduce repeating values.
- ✓ Evaluated once and used multiple times.

3. OUTPUTS

- ★ Outputs display important information after apply.
- ★ Defined using the **output** block.
- ★ Useful to show values like IDs, IPs, DNS names, etc.

Example - outputs.tf

```
output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.web.id
}

output "public_ip" {
  description = "Public IP of the instance"
  value       = aws_instance.web.public_ip
}
```

Example - Output after apply

```
Outputs:
instance_id = "i-0a1b2c3d4e5f6g7h8"
public_ip   = "54.123.45.67"
```

★ Key Points

- ✓ Outputs are shown at the end of apply.
- ✓ Can be used by other modules via **module.<name>.<output_name>**

4. VARIABLES vs LOCALS vs OUTPUTS

Feature	Variables	Locals	Outputs
Purpose	Input values to configure code	Store intermediate values	Show important information
Defined Using	variable block	locals block	output block
Provide By	User (tfvars / CLI / env)	Defined in code	Defined in code
Change During Run	Yes (by user)	No (fixed in code)	No (read-only result)
Used For	Customization	Code reusability & simplification	Information / Integration
Example	var.region	local.common_tags	output.instance_id



Quick Tips

- Use variables for all user inputs.
- Use locals to avoid repeated expressions.
- Use outputs to display values or pass to other modules.

Write Once,
Use Anywhere,
Automate Everything!

T8. TERRAFORM PLAN, APPLY & DESTROY

Terraform follows a simple workflow: **Plan** → **Apply** → **Destroy**

These commands help you preview, create/update and remove infrastructure.



Plan



Apply



Destroy

1. TERRAFORM PLAN

- ★ Plans your changes.
- ★ Shows what resources will be created, updated or destroyed.
- ★ Does **not** make any real changes.
- ★ Generates an execution plan.

Example

```
$ terraform plan
...
# Output (summary)
+ 1 to add
~ 2 to change
- 0 to destroy

Plan: 3 to add, 2 to change, 0 to destroy.
```

★ Key Points

- ✓ Safe to run.
- ✓ Preview before making changes.
- ✓ Use `-out` to save plan to a file.

Cmd: `terraform plan`

2. TERRAFORM APPLY

- ★ Applies the changes required to reach the desired state.
- ★ Creates, updates or deletes resources.
- ★ Asks for confirmation before making changes.

Example

```
$ terraform apply
...
Terraform will perform the following actions:
+ aws_instance.web will be created
~ aws_s3_bucket.data will be updated
Plan: 1 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
Enter a value: yes

Apply complete! Resources: 1 added, 1 changed,
0 destroyed.
```

★ Key Points

- ✓ Actually makes the changes.
- ✓ Use `-auto-approve` to skip confirmation.
- ✓ Always run plan before apply.

Cmd: `terraform apply`

3. TERRAFORM DESTROY

- ★ Destroys all resources managed by Terraform.
- ★ Removes infrastructure.
- ★ Asks for confirmation before deleting.

Example

```
$ terraform destroy
...
Terraform will perform the following actions:
- aws_instance.web will be destroyed
- aws_s3_bucket.data will be destroyed
Plan: 0 to add, 0 to change, 2 to destroy.

Do you really want to destroy all resources?
Enter a value: yes

Destroy complete! Resources: 2 destroyed.
```

★ Key Points

- ✓ Deletes all managed resources.
- ✓ Use with caution.
- ✓ Always backup important data.

Cmd: `terraform destroy`

Workflow Summary



1. Plan

Preview changes
(Safe)



2. Apply

Implement changes
(Makes changes)



3. Destroy

Remove infrastructure
(Use with caution)



Quick Tip

Always run terraform plan before apply to avoid unexpected changes.

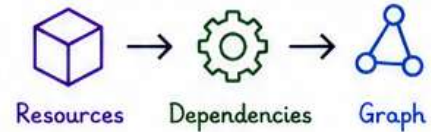
Plan First,
Then Apply with
Confidence!



T9. TERRAFORM DEPENDENCY & RESOURCE GRAPH

Terraform automatically manages dependencies between resources.

It builds a dependency graph to determine the order of creation, update and deletion of resources.



1. DEPENDENCY IN TERRAFORM

- ★ Terraform automatically detects dependencies based on references.
- ★ If Resource A uses Resource B, Terraform knows B must be created first.
- ★ You can also define explicit dependencies using `depends_on`.
- ★ This ensures correct creation, update and deletion order.

Example - Implicit Dependency (Auto-detected)

```

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "public" {
  vpc_id      = aws_vpc.main.id # Reference creates dependency
  cidr_block = "10.0.1.0/24"
}

# Subnet will be created after VPC automatically
  
```

2. EXPLICIT DEPENDENCY

- ★ Use `depends_on` meta-argument when there is no direct reference but still a dependency exists.
- ★ Forces Terraform to follow the specified order.

Example - Using `depends_on`

```

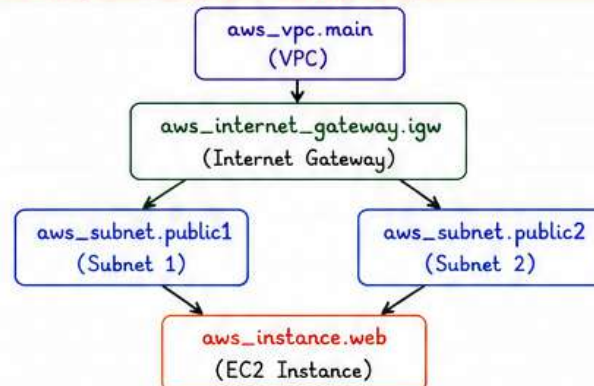
resource "aws_s3_bucket" "bucket" {
  bucket = "my-unique-bucket"
}

resource "aws_s3_bucket_acl" "acl" {
  bucket = aws_s3_bucket.bucket.id
  acl    = "private"
  depends_on = [aws_s3_bucket.bucket] # Explicit dependency
}
  
```

3. RESOURCE GRAPH

- ★ Terraform builds a graph of all resources and their dependencies.
- ★ Graph shows the order in which resources will be created, updated or destroyed.
- ★ Helps in understanding complex infrastructure.

Example - Simple Resource Graph



Legend:

- Depends on
- Depends on
- Depends on
- Final Resource

Terraform will create resources from top to bottom.

4. VIEW RESOURCE GRAPH

You can visualize the graph using the following command:

```
$ terraform graph
| dot -Tpng > graph.png
```

Open `graph.png` to see the graph.

★ Key Points

- ✓ Terraform automatically handles most dependencies.
- ✓ Use `depends_on` only when necessary.
- ✓ Graph helps in troubleshooting and understanding resource order.
- ✓ Resources are created in dependency order and destroyed in reverse order.



Quick Tip

Always check the resource graph for complex setups to ensure correct creation and deletion order.

Plan Smart,
Apply Confidently!

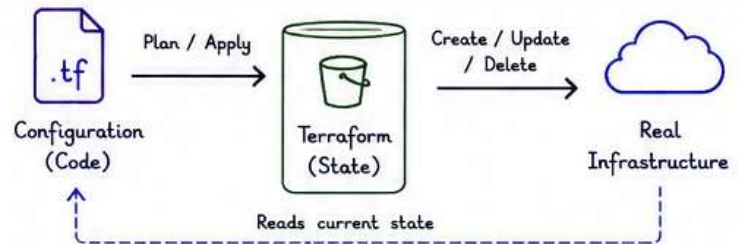


T10. TERRAFORM STATE — CORE CONCEPT

Terraform State is a file that keeps track of the real infrastructure managed by Terraform.

It maps your configuration to real-world resources and stores their current state.

Terraform uses this state to plan, create, update and destroy resources.



1. WHAT IS STATE?

- ★ State is a JSON file.
- ★ It stores information about all resources Terraform manages.
- ★ Includes resource IDs, attributes, metadata and dependencies.
- ★ Without state, Terraform doesn't know what exists in real infrastructure.

Example - Local State File (terraform.tfstate)

```

{
  "version": 4,
  "terraform_version": "1.8.5",
  "resources": [
    {
      "type": "aws_instance",
      "name": "web",
      "instances": [
        {
          "schema_version": 1,
          "attributes": {
            "id": "i-0abc123def4567890",
            "instance_type": "t2.micro",
            "private_ip": "10.0.1.10"
          }
        }
      ]
    }
  ]
}
  
```

★ Key Points

- ✓ State represents the real infrastructure.
- ✓ Terraform compares configuration with state to decide changes.
- ✓ State is critical for accurate operations.
- ✓ Keep state safe and consistent.

2. WHY STATE IS IMPORTANT?

- ★ Enables Terraform to know what resources already exist.
- ★ Helps in generating accurate plans.
- ★ Maintains dependencies between resources.
- ★ Enables updates and deletions correctly.

Without State?

- ✗ Terraform will think nothing exists.
- ✗ It may try to recreate existing resources.
- ✗ Can lead to duplication, errors and downtime.
- ✗ Deletes may not work as expected.

State is the Source of Truth for Terraform



3. STATE BACKENDS (WHERE STATE IS STORED)

- ★ By default, state is stored locally (terraform.tfstate).
- ★ For teams and production, use remote backends.
- ★ Remote state provides: locking, consistency, versioning & collaboration.

Common Remote Backends

- AWS S3 + DynamoDB (locking)
- Azure Blob Storage
- Google Cloud Storage
- Terraform Cloud / Enterprise

Benefits of Remote State

- ✓ State locking (prevents conflicts)
- ✓ Team collaboration
- ✓ Centralized and secure
- ✓ Versioning & backup

4. STATE OPERATIONS (BASIC)

- ★ Init backend to configure where state is stored.
- ★ Pull, push or refresh state as needed.
- ★ Manage state carefully.

Useful Commands

Command	Purpose
terraform init	Initializes backend and downloads state
terraform state list	Lists resources in state
terraform state show <resource>	Shows details of a resource in state
terraform refresh	Refreshes state with real infrastructure
terraform state rm <resource>	Removes a resource from state
terraform import <resource> <id>	Imports existing resource into state

5. SUMMARY

Aspect	Local State	Remote State
Location	Stored on your machine	Stored in remote backend (e.g., S3)
Collaboration	Not suitable for teams	Designed for teams
Locking	Not available	Available (e.g., DynamoDB)
Versioning	Manual	Automatic (backend dependent)
Security	Less secure	More secure



Quick Tip

- Always use remote state in team or production environments.
- Enable state locking to avoid concurrent changes.
- Backup your state regularly.

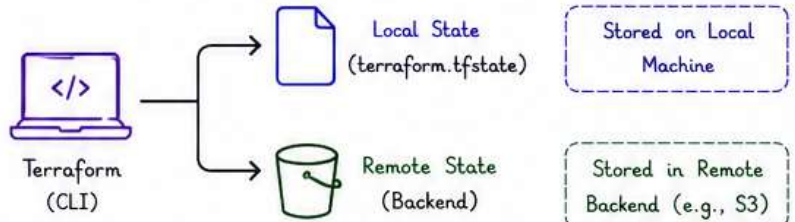
State is Small but Powerful - Handle it with Care!



Terraform State can be stored locally or remotely.

Both have their own pros and cons.

Choosing the right backend is important for team collaboration, security and consistency.



1. LOCAL STATE

- ★ By default Terraform stores state locally in a file named `terraform.tfstate`.
- ★ Stored in the working directory.
- ★ Good for learning, testing and single user.
- ★ Not recommended for teams or production.

Example (Local)

No backend is configured.

State file Location:

`./terraform.tfstate`

☆ Pros

- ✓ Simple to setup
- ✓ No extra infrastructure

☆ Cons

- ✓ Not suitable for teams
- ✓ No locking (risk of conflicts)
- ✓ No versioning / backup

2. REMOTE STATE

- ★ State is stored in a remote backend.
- ★ Enables collaboration and state locking.
- ★ Provides versioning, backup and security.
- ★ Recommended for teams and production.

Example (Remote - AWS S3)

```

terraform {
  backend "s3" {
    bucket     = "my-terraform-state"
    key        = "envs/dev/terraform.tfstate"
    region     = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt    = true
  }
}
  
```

☆ Pros

- ✓ State locking (prevents conflicts)
- ✓ Team collaboration
- ✓ Versioning & backup
- ✓ Centralized and secure

☆ Cons

- ✓ Slightly more setup
- ✓ Cost for backend resources

3. LOCAL vs REMOTE STATE COMPARISON

Feature	Local State	Remote State
Storage Location	Local machine (terraform.tfstate)	Remote backend (e.g., S3, GCS, Azure Blob)
Collaboration	Not suitable	Suitable for teams
State Locking	Not available	Available (e.g., DynamoDB, GCS locking)
Versioning	Not available	Available (e.g., S3 versioning)
Backup	Manual (user responsibility)	Automatic (backend dependent)
Security	Less secure (local access)	More secure (IAM, encryption, access control)
Setup Complexity	Very simple	Requires backend configuration
Best For	Learning, Testing, Single user	Teams, Production

4. COMMON REMOTE BACKENDS

AWS S3 + DynamoDB



- ✓ Most popular
- ✓ State locking with DynamoDB
- ✓ Versioning & encryption
- ✓ Highly available

Azure Blob Storage



- ✓ Secure & scalable
- ✓ Lease based locking
- ✓ Versioning support
- ✓ Good for Azure users

Google Cloud Storage



- ✓ Global & durable
- ✓ Generation based locking
- ✓ Versioning support
- ✓ Good for GCP users

Terraform Cloud / Enterprise



- ✓ Managed by HashiCorp
- ✓ Remote state + runs
- ✓ Built-in locking
- ✓ Team collaboration



Quick Tips

- Always use remote state for team & production.
- Enable state locking to avoid concurrent modification.
- Enable versioning & backup for disaster recovery.
- Use least privilege access for backend resources.

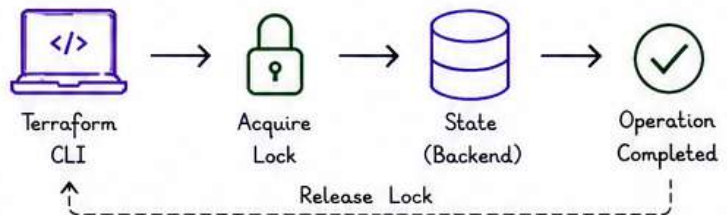
Manage State Smartly,
Avoid Conflicts,
Stay Consistent!



T12. STATE LOCKING & STATE MANAGEMENT

State locking prevents multiple operations from modifying the state file at the same time.

Good state management ensures state file is safe, consistent and easy to maintain.



1. WHY STATE LOCKING?

- ★ Prevents concurrent operations.
- ★ Avoids state corruption.
- ★ Ensures data consistency.
- ★ Required for team collaboration in remote backends.

2. HOW STATE LOCKING WORKS?

- ★ When a Terraform operation starts, it tries to acquire a lock on the state.
- ★ If lock is acquired → operation continues.
- ★ If lock is held by someone else → operation waits or fails.
- ★ After operation completes, the lock is released.



3. LOCKING IN REMOTE BACKENDS

Most remote backends support automatic state locking.

Backend	Locking Mechanism
AWS S3	DynamoDB
Azure Blob	Blob Lease
GCS	GCS Object Lock
Terraform Cloud	Built-in Locking
Consul	Session Lock
Etcd	Lease Lock

If locking is not configured, Terraform may not be able to prevent concurrent changes.

4. EXAMPLE - AWS S3 + DYNAMODB LOCKING

```

terraform {
  backend "s3" {
    bucket     = "my-terraform-state"
    key        = "envs/dev/terraform.tfstate"
    region     = "ap-south-1"
    dynamodb_table = "terraform-locks" # For state locking
    encrypt    = true
  }
}
  
```

How it works:

- State file is stored in S3.
- DynamoDB table is used to acquire and release locks.
- Ensures only one operation can modify the state at a time.

DynamoDB Table Example

LockID (Partition Key)	Info	Expires
my-terraform-state	Lock info (owner, time)	TTL (Optionall)

Terraform creates an item in DynamoDB when lock is acquired and deletes it when lock is released.

5. STATE MANAGEMENT BEST PRACTICES



Enable state locking always.



Use remote backend for teams & prod.



Store state securely with encryption.



Enable versioning & backups.



Restrict access to state backend.



Never edit state file manually.

6. USEFUL COMMANDS FOR STATE MANAGEMENT

Command	Description
<code>terraform state list</code>	Lists all resources in the state.
<code>terraform state show <address></code>	Shows details of a resource in state.
<code>terraform state rm <address></code>	Removes a resource from state (does not delete real resource).
<code>terraform import <address> <id></code>	Imports existing resource into state.
<code>terraform state pull</code>	Downloads current state to stdout.
<code>terraform state push <file></code>	Uploads a state file to backend (use carefully).

Note



Use `state rm` and import carefully.

Always backup state before making changes.

7. WHAT HAPPENS WITHOUT STATE LOCKING?

- ✗ Two users may run operations at the same time.
- ✗ State file can become inconsistent or corrupted.
- ✗ Resources may be created, updated or deleted unexpectedly.
- ✗ Leads to downtime and data loss.

VS

WITH STATE LOCKING

- ✓ Only one operation at a time.
- ✓ State remains consistent.
- ✓ Safe for teams and production.
- ✓ Prevents conflicts and errors.

Key Takeaway



State locking is like a safety lock for your infrastructure. Enable it and manage your state like a critical asset!



Quick Tips

- Always use remote state with locking in team & production.
- Enable versioning & backup for disaster recovery.
- Regularly review who has access to the state backend.
- A well-managed state = reliable & repeatable infrastructure!

Lock the State,
Protect Your
Infrastructure!



A **Backend** defines where and how Terraform stores the state file.

It enables collaboration, state locking, versioning and remote state storage.



Terraform CLI



Configuration (Code)



Backend



State File (Remote)

1. WHAT IS A BACKEND?

- ★ Backend is the location where Terraform stores the state file.
- ★ It is configured in the **backend** block.
- ★ Backend configuration is separate from resources.
- ★ It is initialized using **terraform init**.
- ★ It supports remote storage, locking and versioning.

2. BACKEND vs PROVIDER

Aspect	Backend	Provider
Purpose	Store state file	Interact with Cloud APIs
Configured In	backend block	provider block
Required	Optional (but recommended)	Required
Example	S3, Azure, GCS, Consul	AWS, Azure, GCP, etc.
Change Impact	Requires re-init (if backend changes)	No re-init (only plan/apply)

3. BACKEND BLOCK SYNTAX

Example:

```
terraform {
  backend "s3" {
    bucket     = "my-terraform-state"
    key        = "envs/dev/terraform.tfstate"
    region     = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt    = true
  }
}
```

Notes:

- The backend block goes inside terraform {}.
- It does not support interpolation or variables.
- Run **terraform init** after any change.

4. POPULAR BACKENDS

AWS S3



- ✓ Most widely used
- ✓ DynamoDB for locking
- ✓ Versioning supported
- ✓ Highly durable
- ✓ Secure & scalable

Azure Blob Storage



- ✓ Native Azure integration
- ✓ Blob lease for locking
- ✓ Versioning supported
- ✓ Good for Azure users

Google Cloud Storage



- ✓ Native GCP integration
- ✓ Generation based locking
- ✓ Versioning supported
- ✓ Durable & scalable

Terraform Cloud / Enterprise



- ✓ Managed by HashiCorp
- ✓ Built-in state locking
- ✓ Versioning & audit logs
- ✓ Team collaboration

Consul



- ✓ Session based locking
- ✓ Used in private networks
- ✓ Self-managed solution

5. EXAMPLE - AWS S3 + DYNAMODB BACKEND

```
terraform {
  backend "s3" {
    bucket     = "my-terraform-state"
    key        = "envs/prod/terraform.tfstate"
    region     = "ap-south-1"
    dynamodb_table = "terraform-locks" # For state locking
    encrypt    = true                  # SSE encryption
  }
}
```

Prerequisites:

- S3 bucket must already exist.
- DynamoDB table must already exist with partition key (LockID).
- IAM user/role should have access to S3 and DynamoDB.

6. INITIALIZING BACKEND

- ★ Run the following command to initialize backend:

```
terraform init
```

- ★ Terraform downloads backend plugins and configures remote state.

- ★ If backend config changes:

```
terraform init -reconfigure
```

- ★ To migrate state to new backend:

```
terraform init -migrate-state
```

Important



Backend block cannot use variables or interpolation.

Backend configuration is local and not stored in the state.

7. COMMON BACKEND ARGUMENTS (S3 EXAMPLE)

Argument	Description	Example
bucket	S3 bucket name	"my-terraform-state"
key	Path to state file inside bucket	"envs/dev/terraform.tfstate"
region	AWS region	"ap-south-1"
dynamodb_table	DynamoDB table for state locking	"terraform-locks"
encrypt	Enable server-side encryption	true / false
profile	AWS CLI profile (optional)	"default"

9. USEFUL COMMANDS

Command	Description
terraform init	Initialize backend and download plugins
terraform init -reconfigure	Reinitialize backend (when config changes)
terraform init -migrate-state	Migrate state to new backend
terraform state list	List resources in the state
terraform state pull	Download current state to stdout
terraform state push <file>	Upload state file to backend (use carefully)

8. BACKEND BEST PRACTICES

- ✓ Always use remote backend for teams & production.
- ✓ Enable state locking to avoid concurrent changes.
- ✓ Enable versioning on the storage bucket.
- ✓ Restrict bucket access using IAM policies.
- ✓ Regularly backup the state file.
- ✓ Use least privilege access for backend resources.



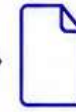
Quick Tip

- Backend = Where state is stored.
- Provider = How resources are managed.
- Use remote backend for safety, team collaboration, locking and versioning.
- Always run **terraform init** first when working with a new backend.

T14. TERRAFORM STATE COMMANDS

Terraform state commands help you inspect, manage and modify the state file.

These commands work directly with the state and do not interact with real infrastructure.

Terraform
CLIState
CommandsState File
(Local / Remote)Manage &
Inspect State

1. COMMON TERRAFORM STATE COMMANDS

COMMAND	PURPOSE	WHEN TO USE
<code>terraform state list</code>	List all resources in the state.	See what is managed in state.
<code>terraform state show <address></code>	Show details of a resource in state.	Inspect resource attributes.
<code>terraform state pull</code>	Pull and display the current state.	View full state in JSON.
<code>terraform state rm <address></code>	Remove a resource from state.	Stop managing a resource (resource remains in infra).
<code>terraform state mv <src> <dst></code>	Move/Rename a resource in state.	Refactor or rename resources.
<code>terraform state import <addr> <id></code>	Import existing resource into state.	Bring existing infra under TF.
<code>terraform state push <file></code>	Push a local state file to backend.	Upload modified state (use carefully).

2. EXAMPLES

List all resources
`terraform state list`

Show details of a resource
`terraform state show aws_instance.web`

Pull current state and save in a file
`terraform state pull > current-state.json`

Remove a resource from state
`terraform state rm aws_s3_bucket.old_bucket`

Rename/Move resource in state
`terraform state mv aws_instance.web aws_instance.app`

Import existing resource
`terraform state import aws_s3_bucket.mybucket my-existing-bucket`

Push local state file to backend
`terraform state push terraform.tfstate`

3. STATE RM vs IMPORT

state rm

- Removes resource from state file.
- Resource remains in real infrastructure.
- Terraform will forget and stop managing it.

state import

- Adds existing resource into state.
- Terraform starts managing it.
- No changes to real infrastructure.

4. IMPORTANT NOTES

- ★ State commands work only on the state file.
- ★ They do not create, update or delete infrastructure.
- ★ Incorrect usage can lead to state mismatch.
- ★ Always backup state before making changes.

5. QUICK SUMMARY

- ✓ Use state commands to manage Terraform state.
- ✓ Helpful for troubleshooting, refactoring and imports.
- ✓ Be careful with "state rm" and "state push".
- ✓ Keep state consistent and backed up.

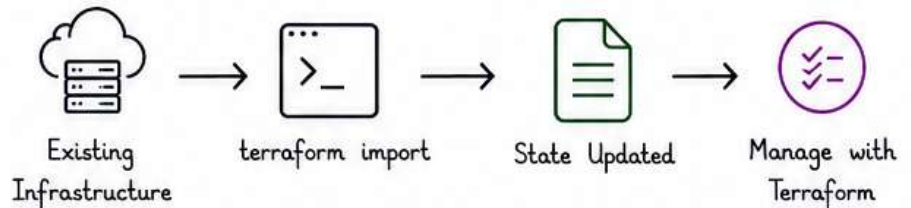


Manage State Smartly,
Keep Infrastructure Safe!



T15. TERRAFORM IMPORT & EXISTING INFRASTRUCTURE

Terraform import lets you bring existing infrastructure (created outside Terraform) under Terraform management without recreating it.



1. WHY IMPORT EXISTING INFRASTRUCTURE?

- ★ Bring manually created resources under Terraform.
- ★ Avoid recreation and downtime.
- ★ Manage all infrastructure from a single place.
- ★ Useful for adoptions and migrations.

2. HOW IMPORT WORKS?

- ① Define the resource in your Terraform configuration (HCL).
- ② Run `terraform import` to map the existing resource to the configuration.
- ③ Terraform updates the state file with the resource.
- ④ Now you can manage it like any Terraform resource.

3. IMPORT COMMAND SYNTAX

```
terraform import <resource_address> <resource_id>
```

`<resource_address>` → Address of the resource in your Terraform configuration.

`<resource_id>` → ID of the existing resource in the cloud provider.

Note

Resource ID format depends on the provider and resource type.

4. EXAMPLE - IMPORT AWS EC2 INSTANCE

HCL Configuration (main.tf)

```
resource "aws_instance" "web" {
  ami           = "ami-0abc123def456"
  instance_type = "t3.micro"
  # other arguments...
}
```

Import Command

```
terraform import aws_instance.web i-0a1b2c3d4e5f6g7h8
```

Here,

- `aws_instance.web` → Resource address
- `i-0a1b2c3d4e5f6g7h8` → Existing EC2 Instance ID

5. STEPS TO IMPORT EXISTING INFRASTRUCTURE

- ① Identify the existing resource ID in your cloud provider.
- ② Add the resource block in Terraform configuration file.
- ③ Run `terraform import <resource_address> <resource_id>`.
- ④ Verify by running `terraform plan`.
- ⑤ Apply changes if needed (optional).

6. THINGS TO REMEMBER

- ★ Import only adds the resource to state, it does not change infrastructure.
- ★ Configuration must match the existing resource.
- ★ Run `terraform plan` after import to check differences.
- ★ Wrong ID or address will cause import failure.

7. COMMON RESOURCES & THEIR ID FORMAT EXAMPLES

Resource	Example ID Format	Example
AWS EC2 Instance	i-xxxxxxxxxxxxxxxxxx	i-0a1b2c3d4e5f6g7h8
AWS S3 Bucket	bucket-name	my-existing-bucket
AWS VPC	vpc-xxxxxxxxxx	vpc-0a1b2c3d4e5f6g7h8
AWS Subnet	subnet-xxxxxxxxxx	subnet-0a1b2c3d4e5f6g7h8
AWS Security Group	sg-xxxxxxxxxx	sg-0a1b2c3d4e5f6g7h8



Quick Tip

Always backup your state file before doing import. It helps to rollback in case of any issue.

Import Once,
Manage Forever!



Terraform Modules are reusable containers of Terraform configuration files (.tf) which can be used to provision infrastructure consistently across environments.



Reusable
Module



Consistent
Infrastructure



Easy to Manage
& Maintain



Use Multiple Times
(Dev, QA, Prod)

1. WHY USE MODULES?

- ★ Reusability - Write once, use multiple times.
- ★ Consistency - Same standard across environments.
- ★ Maintainability - Update in one place, used everywhere.
- ★ Abstraction - Hide complex resources behind simple inputs.
- ★ Collaboration - Teams can share and use modules.

2. MODULE STRUCTURE

my-module/

```

├── main.tf      # Resource definitions
├── variables.tf # Input variables
├── outputs.tf   # Output values
├── versions.tf  # Provider & version info
└── README.md   # Module documentation
  
```

Description

A module is a directory containing Terraform configuration files.

3. MODULE SOURCE

Modules can be sourced from:

- Local path (./my-module)
- Remote Git repository (GitHub, GitLab, etc.)
- Terraform Registry (registry.terraform.io)
- Private registry / custom sources

4. USING A MODULE (EXAMPLE)

```

module "vpc" {
  source      = "../modules/vpc"
  cidr_block = "10.0.0.0/16"
  environment = "dev"
  azs        = ["ap-south-1a", "ap-south-1b"]
}
  
```

Explanation

We are calling a module named "vpc" and passing input values to it.

5. COMMON FILES INSIDE MODULE

main.tf (Resources)

```

resource "aws_vpc" "this" {
  cidr_block = var.cidr_block
  tags = {
    Name = "${var.environment}-vpc"
  }
}
  
```

Contains the actual infrastructure resources.

variables.tf (Inputs)

```

variable "cidr_block" {
  type        = string
  description = "CIDR block for VPC"
}

variable "environment" {
  type = string
}
  
```

Defines input variables for the module.

outputs.tf (Outputs)

```

output "vpc_id" {
  value = aws_vpc.this.id
}

output "vpc_cidr" {
  value = aws_vpc.this.cidr_block
}
  
```

Returns useful information from the module.

versions.tf (Providers)

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = ">= 5.0"
    }
  }
}
  
```

Defines provider and Terraform version.

6. HOW MODULES WORK?

- ① Create module with resources, variables and outputs.
- ② Call the module in root configuration using module block.
- ③ Pass input values (variables) to the module.
- ④ Terraform provisions the resources defined in the module.
- ⑤ Use outputs from the module in other resources if needed.

7. MODULE BEST PRACTICES

- ✓ Keep modules small and focused on a single purpose.
- ✓ Use input variables for flexibility.
- ✓ Provide meaningful outputs.
- ✓ Add descriptions for variables and outputs.
- ✓ Version your modules when used in teams.
- ✓ Test modules before using in production.

8. TERRAFORM REGISTRY

Terraform Registry (registry.terraform.io) provides public modules.

Example:

```

module "s3" {
  source      = "terraform-aws-modules/s3-bucket/aws"
  version    = "~> 4.1"
  bucket     = "my-app-bucket"
  acl       = "private"
}
  
```

Benefits:

- ✓ Pre-built and tested modules
- ✓ Easy to use
- ✓ Saves development time



Quick Tip

Modules make your Terraform code reusable, clean, and easy to maintain at scale!

T17. MODULE STRUCTURE & BEST PRACTICES

Topic : Terraform

Date : 01/08/2026

@ codewithanush

Terraform Modules help you organize, reuse and manage infrastructure as code efficiently. A good module structure and best practices make your code clean, scalable and easy to maintain.

Module
StructureReusable
CodeEasy to Manage
& MaintainBest Practices
Followed

1. STANDARD MODULE STRUCTURE

my-module/

```

├── main.tf           # Resource definitions
├── variables.tf       # Input variables
├── outputs.tf         # Output values
├── versions.tf        # Provider & version info
├── README.md          # Module documentation
├── locals.tf          # Local values (optional)
└── terraform.tfvars.example # Example inputs (optional)

```

File Purpose

- main.tf - Contains the actual infrastructure resources.
- variables.tf - Defines inputs to the module.
- outputs.tf - Returns values from the module.
- versions.tf - Declares required providers and versions.
- README.md - Explains usage, inputs & outputs.
- terraform.tfvars.example - Sample input values.
- locals.tf - Defines local values for reuse within module.

2. VARIABLES & OUTPUTS EXAMPLE

variables.tf (Inputs)

```

variable "cidr_block" {
  type        = string
  description = "CIDR block for VPC"
}

variable "environment" {
  type        = string
  description = "Environment name"
  default     = "dev"
}

```

outputs.tf (Outputs)

```

output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.this.id
}

output "public_subnet_ids" {
  description = "List of public subnet IDs"
  value       = aws_subnet.public[*].id
}

```

Note:

- Variables make modules flexible.
- Outputs expose important values to the calling configuration.

3. USING A MODULE (EXAMPLE)

```

module "vpc" {
  source      = "../modules/vpc"
  cidr_block  = "10.0.0.0/16"
  environment = "dev"
  azs         = ["ap-south-1a", "ap-south-1b"]
}

```

Explanation:

We are calling a module named "vpc" from local path and passing input values to it.

4. MODULE BEST PRACTICES

- ★ Keep modules small and focused on a single purpose.
- ★ Use input variables for all configurable values.
- ★ Provide meaningful descriptions for variables and outputs.
- ★ Follow consistent naming conventions.
- ★ Keep modules idempotent and environment independent.
- ★ Avoid hardcoding values.
- ★ Version your modules (using Git tags or Registry).
- ★ Write README with usage examples.
- ★ Test modules before using in production.

5. DO'S & DON'TS

DO'S

- ✓ Use variables for dynamic values.
- ✓ Return only necessary outputs.
- ✓ Keep module structure standard.
- ✓ Document modules properly.
- ✓ Use latest provider versions.

DON'TS

- ✗ Don't hardcode account/region.
- ✗ Don't expose sensitive values.
- ✗ Don't create circular dependencies.
- ✗ Don't make modules too generic.
- ✗ Don't ignore versioning.

6. LOCALS (OPTIONAL)

```

locals {
  common_tags = {
    Project      = "MyApp"
    Environment  = var.environment
    ManagedBy    = "Terraform"
  }
}

```

Use locals to store values that are reused within the module.

7. MODULE BEST PRACTICES SUMMARY

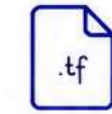
Area	Best Practice
Reusability	Design modules to be reusable across projects and environments.
Flexibility	Expose inputs as variables with sensible defaults.
Consistency	Follow standard file structure and naming conventions.
Maintainability	Keep modules simple, well documented and versioned.
Security	Avoid exposing sensitive information through outputs.
Testing	Validate modules using <code>terraform validate</code> and <code>terraform plan</code> .
Collaboration	Publish modules in private registry or version control.

Quick Tip

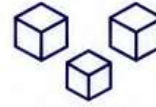
Good modules are the building blocks of infrastructure automation. Write once, use everywhere!

Both `count` and `for_each` are meta-arguments used to create multiple instances of a resource.

The main difference is how they track and identify those instances.



Terraform
Config



Multiple
Instances



Managed by
Terraform

1. COUNT vs FOR_EACH - AT A GLANCE

Feature	count	for_each
Purpose	Create multiple instances based on a number.	Create multiple instances based on a collection (set or map).
Input Type	Number	Map or Set of strings
Instance Index	Uses numeric index (0, 1, 2, ...)	Uses key of the map/set
Address Format	<code>resource.type.name[count.index]</code>	<code>resource.type.name["key"]</code>
Order Dependency	Yes - Changing order can recreate resources.	No - Keys keep resources stable.
Best Use Case	When you need N similar resources.	When you have named/unique items (stable identity).
Example Input	<code>count = 3</code>	<code>for_each = { web = "t3.micro", db = "t3.small" }</code>
Instance Change Impact	Adding/removing in middle may recreate others.	Only added/removed key is affected.

2. COUNT EXAMPLE

```
resource "aws_instance" "web" {
  count      = 3
  ami       = "ami-0abc123def456"
  instance_type = "t3.micro"
  tags = {
    Name = "web-${count.index}"
  }
}
```

Creates 3 instances with names:
web-0, web-1, web-2

3. FOR_EACH EXAMPLE

```
resource "aws_instance" "server" {
  for_each = {
    web = "t3.micro"
    db  = "t3.small"
    app = "t3.medium"
  }
  ami       = "ami-0abc123def456"
  instance_type = each.value
  tags = {
    Name = each.key
  }
}
```

Creates 3 instances with names:
web, db, app

4. WHEN TO USE WHAT?

USE COUNT WHEN

- You need multiple similar resources.
- The number of instances is fixed or comes from a variable.
- You don't need unique names or keys for each instance.

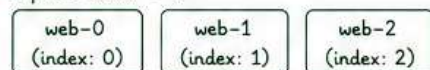
USE FOR_EACH WHEN

- You have a list/set/map of items with unique keys.
- You want stable identity even if items are added/removed.
- You need to reference instances by meaningful names.

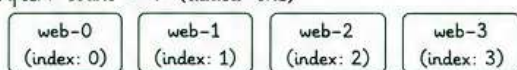
5. KEY DIFFERENCES WITH EXAMPLE

COUNT (Index Based)

Before: `count = 3`



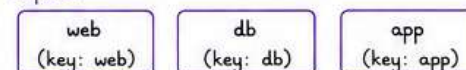
After: `count = 4` (added one)



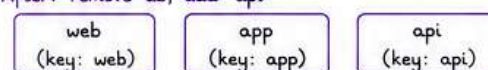
- If you remove `web-1`, then `web-2` becomes `web-1`.
→ Terraform will recreate affected resources.

FOR_EACH (Key Based)

Before:



After: remove db, add api



- Only `"db"` is destroyed.
- New `"api"` is created. Others remain same.



Tip

Use `for_each` whenever possible for better stability and maintainability.

6. IMPORTANT NOTES

- ★ `count.index` starts from 0.
- ★ `for_each` keys must be known at plan time.
- ★ You cannot use both `count` and `for_each` in the same resource.
- ★ `for_each` does not support list with duplicate values (use `toset()`).
- ★ Always prefer `for_each` for production-grade configurations.

7. QUICK SUMMARY

- ✓ `count` → for number of similar resources.
- ✓ `for_each` → for named, unique resources.
- ✓ `for_each` provides stable identity.
- ✓ Better code = easier to manage!



TERRAFORM

T19. CONDITIONAL EXPRESSIONS & FUNCTIONS

Topic : Terraform

Date : 01/08/2026

@ codewithanush

Conditional expressions and functions help you write dynamic, reusable and smarter Terraform code.

Terraform
ConfigConditional
Expressions

Functions

Smarter &
Dynamic Code

1. CONDITIONAL EXPRESSIONS (TERNARY OPERATOR)

Syntax

```
condition ? true_value : false_value
```

Example

```
var.environment == "prod" ? "t3.large" : "t3.micro"
```

↓ ↓ ↓ ↓

Condition If TRUE If If FALSE

Use Cases

- Select values based on conditions.
- Set different configurations for environments.
- Avoid duplicating resources.

2. USING CONDITIONAL EXPRESSION (EXAMPLE)

```
variable "environment" {
  type    = string
  default = "dev"
}

resource "aws_instance" "web" {
  ami           = "ami-0abc123def456"
  instance_type = var.environment == "prod"
    ? "t3.large"
    : "t3.micro"

  tags = {
    Name = "web-${var.environment}"
  }
}
```

If environment
is "prod" →
t3.large
otherwise →
t3.micro

3. COMMONLY USED FUNCTIONS

CATEGORY	FUNCTION	SYNTAX / EXAMPLE	DESCRIPTION
STRING	upper()	upper("hello") => "HELLO"	Converts string to uppercase.
	lower()	lower("HELLO") => "hello"	Converts string to lowercase.
	title()	title("hello world") => "Hello World"	Converts first letter of each word to uppercase.
	trim()	trim(" hello ") => "hello"	Removes leading and trailing spaces.
	substr()	substr("abcdef", 0, 3) => "abc"	Returns substring (start index, length).
COLLECTION	length()	length([1,2,3]) => 3	Returns number of items in list/map/string.
	contains()	contains(["a","b"], "a") => true	Checks if value exists in list.
	lookup()	lookup({a="1", b="2"}, "a", "0") => "1"	Returns value of key or default if not found.
	element()	element(["a","b","c"], 1) => "b"	Returns element at given index.
TYPE	tostring()	tostring(123) => "123"	Converts to string.
	tonumber()	tonumber("123") => 123	Converts to number.
	tolist()	tolist(set("a","b")) => ["a","b"]	Converts set to list.
	tomap()	tomap({a="1", b="2"}) => {a="1", b="2"}	Converts object to map.



Quick Tip

Functions make your code flexible, reduce manual work and handle complex logic easily.

4. FUNCTIONS IN PRACTICE (EXAMPLE)

```
variable "name" {
  default = "my-app"
}

locals {
  app_name = trim(var.name)
  upper_name = upper(local.app_name)
  name_length = length(local.app_name)
  is_prod = contains(["prod", "prd"], "prod")
}
```

Output

```
• local.app_name => "my-app"
• local.upper_name => "MY-APP"
• local.name_length => 6
• local.is_prod => true
```

5. NESTED CONDITIONAL EXAMPLE

```
locals {
  instance_type = var.environment == "prod" ? "t3.large" :
    var.environment == "stage" ? "t3.medium" :
    "t3.micro"
}
```

Explanation

- If prod → t3.large
- Else if stage → t3.medium
- Else → t3.micro

6. BEST PRACTICES

- ✓ Use conditional expressions for dynamic values.
- ✓ Prefer functions to avoid hardcoding.
- ✓ Keep expressions simple and readable.
- ✓ Test with terraform plan to validate results.

7. IMPORTANT NOTES

- ★ Conditional expression always returns one of the two values.
- ★ Functions do not cause side effects, they only transform data.
- ★ Use functions in locals for cleaner and reusable code.
- ★ Not all functions work with all data types, so check documentation.

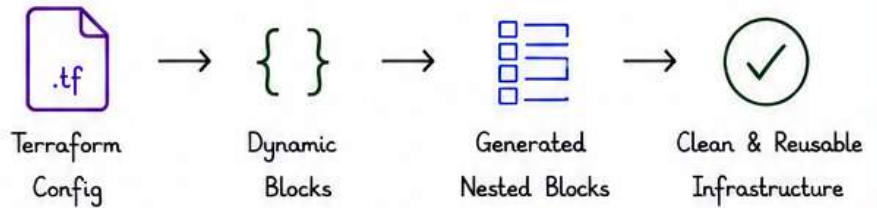
8. COMMONLY USED LOGICAL FUNCTIONS

- ! (not) → Logical NOT
- && (and) → Logical AND
- || (or) → Logical OR
- Can be used inside conditional expressions.

Example

```
var.environment == "prod" &&
var.enable_monitoring
? true : false
→ True only if both are true
```

Dynamic blocks help you generate repeatable nested blocks dynamically based on a collection (list/map/set). This makes your code DRY, flexible and easier to maintain.



1. WHAT IS A DYNAMIC BLOCK?

- Used to dynamically generate repeatable nested blocks inside a resource.
- Works with list, set or map.
- Reduces duplication and improves readability.
- The content {} defines what goes inside each generated block.

2. SYNTAX

```

dynamic "<label>" {
  for_each = <collection>
  iterator = <iterator_name> # (optional)
  content {
    <arguments using iterator>
  }
}
  
```

<label> → The nested block label to generate

<collection> → List/Set/Map to iterate over

<iterator_name> → Name used to access each item

content {} → Body of the nested block

3. EXAMPLE - AWS SECURITY GROUP INGRESS RULES

VARIABLE

```

variable "ingress_rules" {
  description = "List of ingress rules"
  type = list(object({
    from_port = number
    to_port   = number
    protocol  = string
    cidr_blocks = list(string)
  }))
  default = [
    { from_port = 80, to_port = 80, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"] },
    { from_port = 443, to_port = 443, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"] }
  ]
}
  
```

RESOURCE USING DYNAMIC BLOCK

```

resource "aws_security_group" "example" {
  name        = "example-sg"
  description = "SG with dynamic ingress rules"
  vpc_id      = "vpc-0abc123def4567890"

  dynamic "ingress" {
    for_each = var.ingress_rules
    iterator = rule
    content {
      from_port = rule.value.from_port
      to_port   = rule.value.to_port
      protocol  = rule.value.protocol
      cidr_blocks = rule.value.cidr_blocks
    }
  }
}
  
```

Result:

- ✓ Creates one ingress block for each item in ingress_rules.
- ✓ In above example, 2 ingress blocks will be generated.

Ingress Rules Created:

- 80/tcp from 0.0.0.0/0
- 443/tcp from 0.0.0.0/0

4. DYNAMIC BLOCK WITH MAP

Example variable (map of descriptions)

```

variable "tags" {
  type = map(string)
  default = {
    Environment = "dev"
    Project     = "my-app"
    Owner       = "terraform"
  }
}
  
```

Usage

```

dynamic "tag" {
  for_each = var.tags
  iterator = t
  content {
    key = t.key
    value = t.value
  }
}
  
```

Note

- For map: iterator.key → key, iterator.value → value
- Default iterator name is label, but it's good practice to set a custom one.

5. WHEN TO USE DYNAMIC BLOCKS

- ✓ When nested blocks are repeatable and count is not known.
- ✓ When the number of nested blocks depends on input variables.
- ✓ To keep code DRY (Don't Repeat Yourself).
- ✓ When using resources that support multiple nested blocks like:
 - security group ingress/egress
 - aws_lb listener
 - aws_route_table route
 - aws_s3_bucket lifecycle_rule
 - etc.

6. DYNAMIC vs COUNT / FOR_EACH

Feature	Dynamic Block	count / for_each
Purpose	Generate nested blocks	Create multiple resources
Works Inside	Resource block	Top-level resource block
Input Type	List / Set / Map	List / Set / Map
Creates	Nested blocks	Separate resources
Use Case	Repeatable config inside a resource	Multiple similar resources

7. IMPORTANT NOTES

- ★ The collection in for_each must be known at plan time.
- ★ Dynamic block cannot be used for meta-arguments like lifecycle, provisioner, depends_on.
- ★ Avoid complex logic inside content {}.
- ★ Use meaningful iterator names for readability.

8. BEST PRACTICES

- ✓ Use dynamic blocks to handle variable number of nested blocks.
- ✓ Keep content {} simple and readable.
- ✓ Validate input variables properly.
- ✓ Test with different inputs using terraform plan.



Quick Tip

Dynamic blocks save a lot of code when you have many similar nested blocks. They make your Terraform code clean, reusable and scalable!

T21. LIFECYCLE RULES & RESOURCE MANAGEMENT

IMPORTANT NOTES

Lifecycle rules allow you to control how Terraform creates, updates and destroys resources. They help prevent accidental changes, downtime and data loss.



Terraform
Config



Lifecycle
Rules



Resource
Management



Safe, Controlled
Infrastructure

1. WHAT ARE LIFECYCLE RULES?

- Lifecycle meta-argument controls how Terraform handles changes to a resource.
- Defined inside the resource block using `lifecycle { }`.
- Helps in managing updates, replacements and deletions safely.
- Useful for production environments to avoid unwanted downtime.

2. SYNTAX

```
resource "aws_instance" "example" {
  ami           = "ami-0abc123def456"
  instance_type = "t3.micro"

  lifecycle {
    # lifecycle rules go here
  }
}
```

Note:

Lifecycle block contains settings that influence how Terraform manages this resource during plan, apply and destroy.

3. LIFECYCLE RULES (ARGUMENTS)

Argument	Description	Use Case	Example
<code>create_before_destroy</code>	Creates the new resource before destroying the old one.	Zero downtime deployments for critical resources (ALB, ASG, RDS, etc.)	<code>lifecycle { create_before_destroy = true }</code>
<code>prevent_destroy</code>	Prevents Terraform from destroying the resource.	Protect critical resources from accidental deletion (S3, RDS, EBS, etc.)	<code>lifecycle { prevent_destroy = true }</code>
<code>ignore_changes</code>	Ignores changes to specified attributes.	Ignore changes made outside Terraform (manual changes).	<code>lifecycle { ignore_changes = ["tags", "user_data"] }</code>
<code>replace_triggered_by</code>	Forces replacement of the resource when the specified values change.	Replace resource when dependent values change (AMI, Launch Template, etc.)	<code>lifecycle { replace_triggered_by = [var.ami_id] }</code>

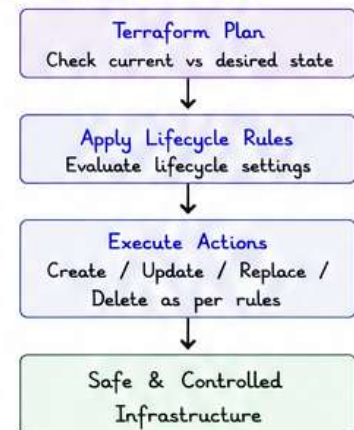
4. WHEN TO USE WHAT?

- `create_before_destroy`
 - When downtime is not acceptable.
 - For ALB, ASG, RDS, ECS services.
- `prevent_destroy`
 - For critical data resources (S3, RDS, EBS).
 - To avoid accidental deletion.
- `ignore_changes`
 - When some attributes are managed outside Terraform.
 - For tags, user_data, IAM policies updated by other teams.
- `replace_triggered_by`
 - When change in a value should force full replacement.
 - For AMI changes, security group changes, etc.

5. BEST PRACTICES

- ✓ Use lifecycle rules only when necessary.
- ✓ Document why a rule is applied.
- ✓ Use `create_before_destroy` for zero downtime.
- ✓ Protect critical data with `prevent_destroy`.
- ✓ Avoid ignoring important changes.
- ✓ Review lifecycle rules periodically.
- ✓ Test in lower environments first.
- ✓ Do not overuse `ignore_changes` (it can hide real drift).

6. LIFECYCLE FLOW



QUICK TIP

Lifecycle rules give you control over Terraform's behavior. Use them wisely to keep your infrastructure safe, stable and reliable!

SUMMARY

- ★ Lifecycle { } manages resource create, update & destroy.
- ★ Use the right rule for the right scenario.
- ★ Fewer surprises, more predictable infrastructure!

TERRAFORM

T22. EXPLICIT DEPENDENCIES & DEPENDS_ON

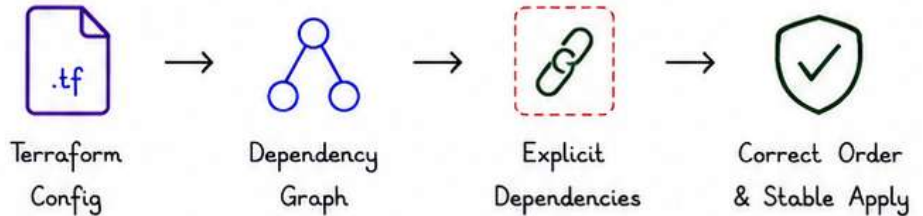
Topic : Terraform

Date : 01/08/2026

@ codewithanush

IMPORTANT NOTES

Terraform automatically builds a dependency graph by analyzing references. In some cases, you need to create explicit dependencies using `depends_on`.



1. WHAT IS DEPENDS_ON?

- `depends_on` is a meta-argument used to create explicit dependencies between resources or modules.
- It tells Terraform: "Create/Update this only after the specified resource(s)."
- By default, Terraform detects dependencies automatically from references.
- Use `depends_on` only when there is no direct reference but order still matters.

2. SYNTAX

```

resource "resource_type" "name" {
  # arguments

  depends_on = [
    resource_type.resource_name,
    another_resource.example
  ]
}
  
```

Note:

`depends_on` accepts a list of resources or modules. It enforces creation, update and destroy order.

3. WHEN TO USE DEPENDS_ON?

- When resources don't have direct references but still have a hidden dependency.
- When using provisioners (local-exec, remote-exec).
- When one resource depends on an external system or side-effect.
- When managing resources across modules.

Avoid using `depends_on`:

- ✗ For every resource.
- ✗ To fix errors in design.
- ✗ Instead of proper references.

4. EXAMPLES

Example 1: S3 Bucket depends on IAM Role

```

resource "aws_iam_role" "example" {
  name = "my-role"
  assume_role_policy = data.aws_iam_policy_document.assume.json
}

resource "aws_s3_bucket" "bucket" {
  bucket = "my-bucket-123"
  depends_on = [aws_iam_role.example]
}
  
```

Example 2: EC2 depends on Security Group

```

resource "aws_security_group" "sg" {
  name = "my-sg"
  # rules ...
}

resource "aws_instance" "web" {
  ami = "ami-0abc123def456"
  instance_type = "t3.micro"
  vpc_security_group_ids = [aws_security_group.sg.id]
  depends_on = [aws_security_group.sg] # Explicit dependency
}
  
```

5. REAL LIFE SCENARIO

You are creating an EC2 instance which requires an IAM instance profile. Even if Terraform can infer some dependencies, using `depends_on` ensures:

- ✓ Correct creation order
- ✓ Fewer intermittent errors
- ✓ Stable deployments
- ✓ Easier troubleshooting

6. BEST PRACTICES

- ✓ Prefer implicit dependencies (via references).
- ✓ Use `depends_on` only when absolutely required.
- ✓ Keep the list minimal but complete.
- ✓ Improves stability in modules and complex environments.
- ✓ Document why `depends_on` is used.

7. KEY TAKEAWAYS

- ★ Terraform builds a dependency graph automatically.
- ★ `depends_on` is used to define explicit dependencies.
- ★ Use it only when Terraform cannot understand the dependency.
- ★ Helps in controlling resource order and avoiding race conditions.
- ★ Overuse can make code harder to maintain.



QUICK TIP

Think twice before adding `depends_on`. If you can create a reference, do that. References are better than explicit dependencies!

DEPENDENCY ORDER FLOW

Plan Phase	Terraform builds dependency graph
↓	
Apply Phase	Resources created in correct order
↓	
Destroy Phase	Resources destroyed in reverse order

Workspaces allow you to manage multiple instances of infrastructure using the same configuration.

Each workspace has its own separate state file.



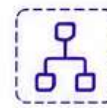
Terraform
Config



Terraform



Provider



Workspaces
(State Isolation)



Multiple
Environments

1. WHAT ARE WORKSPACES?

- Workspaces provide a way to manage multiple environments (dev, test, prod) using the same Terraform configuration.
- Each workspace has its own Terraform state.
- Default workspace is 'default'.
- Other workspaces are user-defined.

2. WHY USE WORKSPACES?

- Manage multiple environments with the same code.
- Keep state files isolated.
- Switch between environments easily.
- Reduce code duplication.
- Useful for Dev, Test, Staging, Prod environments.

3. COMMON WORKSPACE COMMANDS

Command	Description	Example
<code>terraform workspace list</code>	List all workspaces	<code>terraform workspace list</code>
<code>terraform workspace new <name></code>	Create a new workspace	<code>terraform workspace new dev</code>
<code>terraform workspace select <name></code>	Switch to an existing workspace	<code>terraform workspace select dev</code>
<code>terraform workspace show</code>	Show current workspace	<code>terraform workspace show</code>
<code>terraform workspace delete <name></code>	Delete a workspace	<code>terraform workspace delete test</code>
<code>terraform workspace select default</code>	Switch to default workspace	<code>terraform workspace select default</code>

4. EXAMPLE WORKFLOW

1. List existing workspaces
`terraform workspace list`
2. Create a new workspace
`terraform workspace new dev`
3. Switch to the new workspace
`terraform workspace select dev`
4. Run Terraform commands
`terraform plan`
`terraform apply`
5. Switch to another workspace
`terraform workspace select prod`
6. Delete a workspace (if not needed)
`terraform workspace delete dev`

5. HOW WORKSPACES STORE STATE?

- Each workspace has its own state file.
- State files are stored in the backend (S3, Remote Backend).
- Default workspace state is stored in a file named:
`terraform.tfstate`
- Other workspaces state files are stored in:
`terraform.tfstate.d/<workspace_name>/terraform.tfstate`
- Example:

```

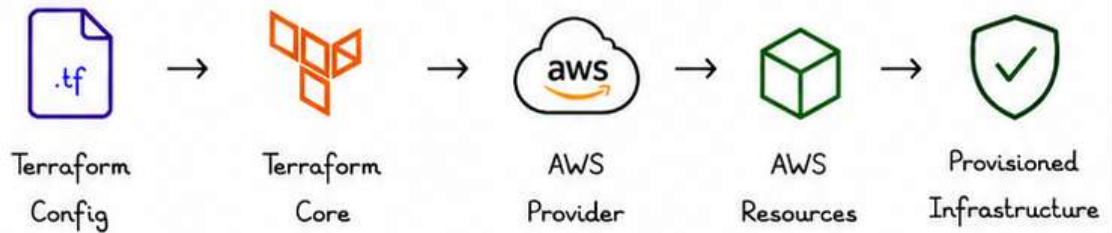
terraform.tfstate.d/
├── dev/
│   └── terraform.tfstate
├── test/
│   └── terraform.tfstate
└── prod/
    └── terraform.tfstate
  
```

6. DEFAULT VS OTHER WORKSPACES

Feature	Default Workspace	Other Workspaces
Name	Always named 'default'	User-defined (e.g., dev, test, prod)
State File	<code>terraform.tfstate</code>	<code>terraform.tfstate.d/<name>/terraform.tfstate</code>
Switching	Cannot be deleted	Can be created, switched, deleted
Use Case	Initial environment	Multiple isolated environments

24. TERRAFORM + AWS PROVIDER

Terraform uses providers to interact with cloud platforms like AWS. The AWS provider allows Terraform to manage AWS resources.



1. WHY AWS PROVIDER?

- Allows Terraform to create and manage AWS resources.
- Communicates with AWS APIs.
- Supports all AWS services.
- Handles authentication, region, and configuration.

2. REQUIRED VARIABLES

- `region` → AWS region where resources will be created.
- `access_key` → AWS Access Key ID.
- `secret_key` → AWS Secret Access Key.
- `token` → Session token (temporary credentials, optional).
- `profile` → AWS CLI profile (optional).

3. AWS PROVIDER CONFIGURATION

```

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region      = "ap-south-1"
  access_key  = "AKIAxxxxxxxxxxxx"
  secret_key  = "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
  # token     = "IQoJb3Jp2GiX2VjE..." (optional)
  # profile   = "default" (optional)
}
  
```

- `required_providers` → Declares the AWS provider and version.
- `source` → Official provider from HashiCorp.
- `version` → Provider version constraint.
- `provider "aws"` → Configures how Terraform connects to AWS.
- `region` → Default region for all resources.
- `access_key`, `secret_key` → Credentials for authentication.
- `profile` → Uses credentials from AWS CLI config file.
- `token` → Used for temporary session credentials.

Note: It is recommended to use environment variables or AWS CLI profile instead of hardcoding credentials.

4. AUTHENTICATION METHODS

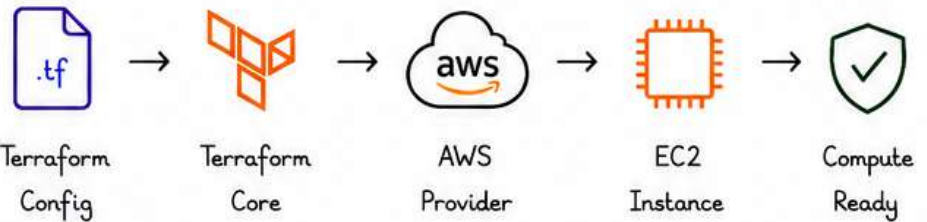
Method	Description
Access Key / Secret Key	Provide <code>access_key</code> and <code>secret_key</code> in provider block.
Environment Variables	Set <code>AWS_ACCESS_KEY_ID</code> and <code>AWS_SECRET_ACCESS_KEY</code> .
AWS CLI Profile	Use <code>profile = "<profile_name>"</code> to load from <code>~/.aws/credentials</code> .
IAM Role (EC2)	If running on EC2 with IAM role attached, no keys needed.
Temporary Credentials	Provide <code>token</code> along with <code>access_key</code> and <code>secret_key</code> .

5. COMMON PROVIDER ARGUMENTS

Argument	Description
<code>region</code>	AWS region (e.g., ap-south-1).
<code>access_key</code>	AWS Access Key ID.
<code>secret_key</code>	AWS Secret Access Key.
<code>token</code>	Session Token for temporary credentials.
<code>profile</code>	AWS CLI profile name.
<code>shared_credentials_file</code>	Path to AWS credentials file.
<code>skip_credentials_validation</code>	Skip credentials validation (true/false).
<code>skip_region_validation</code>	Skip region validation (true/false).
<code>default_tags</code>	Set default tags for all resources.

EC2 (Elastic Compute Cloud) provides resizable compute capacity in the AWS Cloud.

Terraform helps to provision and manage EC2 instances easily.



1. WHAT IS EC2?

- EC2 allows you to run virtual machines in the cloud.
- You can choose instance type, AMI, storage, networking, and security.
- Pay for what you use.
- Highly scalable and available.

2. REQUIRED PROVIDER

```
provider "aws" {
  region = "ap-south-1"
}
```

- Configure AWS provider with your region.
- Make sure AWS CLI is configured with valid credentials.

3. CREATE AN EC2 INSTANCE

```
resource "aws_instance" "my_ec2" {
  ami           = "ami-0c55b159cbfate1f0" # Amazon Linux 2 AMI
  instance_type = "t2.micro"               # Free tier eligible
  key_name      = "my-key"
  subnet_id    = "subnet-0a1b2c3d4e5f67890"
  vpc_security_group_ids = ["sg-0a1b2c3d4e5f67890"]

  tags = {
    Name = "MyEC2Instance"
  }
}
```

Explanation:

- **ami** → AMI ID to launch the instance.
- **instance_type** → Type of EC2 instance.
- **key_name** → Key pair to connect via SSH.
- **subnet_id** → Subnet where instance will run.
- **vpc_security_group_ids** → Security group for instance.
- **tags** → Name/metadata for the instance.

4. CREATE A SECURITY GROUP

```
resource "aws_security_group" "my_sg" {
  name        = "allow_ssh_http"
  description = "Allow SSH and HTTP"
  vpc_id      = "vpc-0a1b2c3d4e5f67890"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # SSH
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # HTTP
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"] # All outbound
  }

  tags = {
    Name = "allow_ssh_http"
  }
}
```

5. CREATE A KEY PAIR (MANUAL STEP)

1. Go to AWS Console → EC2 → Key Pairs → Create Key Pair.
2. Enter key name: my-key
3. Download the .pem file and keep it safe.
4. Use this key to connect to EC2 via SSH.

6. OUTPUT PUBLIC IP

```
output "ec2_public_ip" {
  description = "Public IP of EC2 Instance"
  value       = aws_instance.my_ec2.public_ip
}

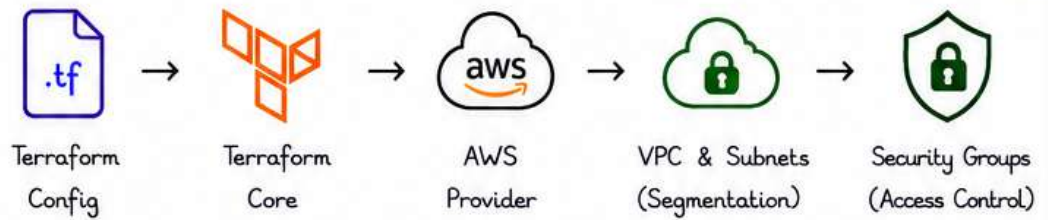
output "ec2_public_dns" {
  description = "Public DNS of EC2 Instance"
  value       = aws_instance.my_ec2.public_dns
}
```

7. COMMON COMMANDS

- **terraform init** → Initialize configuration.
- **terraform plan** → Preview changes.
- **terraform apply** → Create resources.
- **terraform destroy** → Destroy resources.
- **terraform output** → Show output values.

26. TERRAFORM – VPC, SUBNETS & SECURITY GROUPS

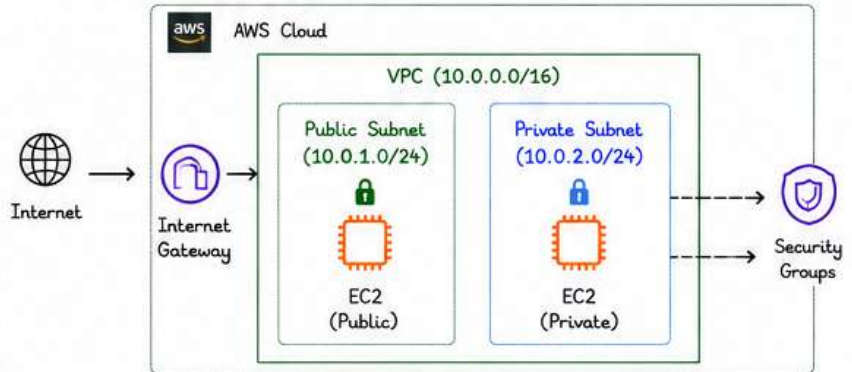
VPC provides a logically isolated network. Subnets exist inside VPC (public / private). Security Groups act as virtual firewalls to control inbound and outbound traffic.



1. WHAT IS VPC?

- Your own virtual network in AWS.
- You define IP address ranges.
- Subnets divide VPC into smaller networks.
- Internet Gateway allows internet access.
- Route Tables control traffic routing.
- Security Groups control traffic to resources.

2. ARCHITECTURE OVERVIEW



3. CREATE VPC

```

resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true

  tags = {
    Name = "main-vpc"
  }
}
  
```

Argument	Description
<code>cidr_block</code>	CIDR block for the VPC.
<code>enable_dns_support</code>	Enables DNS resolution in the VPC.
<code>enable_dns_hostnames</code>	Enables DNS hostnames for instances.
<code>tags</code>	Name/metadata for the VPC.

4. CREATE SUBNETS

```

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "ap-south-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "public-subnet"
  }
}

resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.2.0/24"
  availability_zone  = "ap-south-1b"
  map_public_ip_on_launch = false

  tags = {
    Name = "private-subnet"
  }
}
  
```

Argument	Description
<code>vpc_id</code>	VPC in which subnet will be created.
<code>cidr_block</code>	CIDR block of the subnet.
<code>availability_zone</code>	AZ for the subnet.
<code>map_public_ip_on_launch</code>	Auto-assign public IP (only for public subnet).
<code>tags</code>	Name/metadata for the subnet.

5. INTERNET GATEWAY & ROUTE TABLE

```

# Internet Gateway
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "igw"
  }
}

# Route Table
resource "aws_route_table" "public-rt" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }

  tags = {
    Name = "public-route-table"
  }
}

# Associate Route Table with Public Subnet
resource "aws_route_table_association" "public-rt-assoc" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public-rt.id
}
  
```

6. SECURITY GROUP

```

resource "aws_security_group" "allow_web" {
  name        = "allow-web"
  description = "allow HTTP and SSH"
  vpc_id      = aws_vpc.main.id

  # Ingress Rules
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # SSH
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # HTTP
  }

  # Egress Rule (All Outbound)
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"] # All Outbound
  }

  tags = {
    Name = "allow-web-sg"
  }
}
  
```

Block	Purpose
<code>ingress</code>	Rules for inbound traffic.
<code>egress</code>	Rules for outbound traffic.
<code>cidr_blocks</code>	IP range allowed.
<code>protocol</code>	tcp/udp/-1 (all protocols).
<code>from_port / to_port</code>	Port range allowed.

7. SUMMARY OF RESOURCES

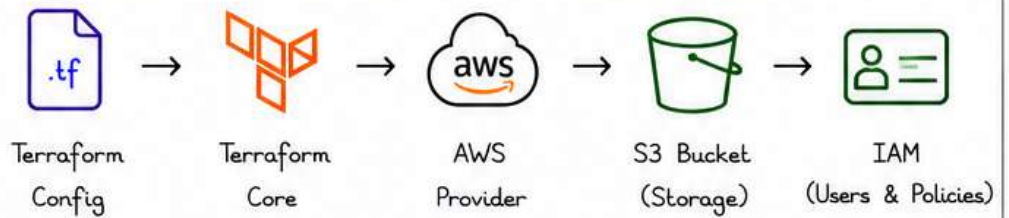
Resource	Purpose
<code>aws_vpc</code>	Creates a Virtual Private Cloud.
<code>aws_subnet</code>	Creates subnets inside the VPC.
<code>aws_internet_gateway</code>	Allows internet access to VPC.
<code>aws_route_table</code>	Defines routing rules.
<code>aws_route_table_association</code>	Associates route table with subnet.
<code>aws_security_group</code>	Controls inbound and outbound traffic.

TERRAFORM

27. TERRAFORM — S3 & IAM

Topic : Terraform
Date : 01/08/2026
@ codewithanush

S3 (Simple Storage Service) is an object storage service in AWS.
IAM (Identity and Access Management) helps you manage users, groups and permissions securely.



1. WHAT IS S3?

- Object storage service to store any amount of data.
- Highly durable, available and scalable.
- Used for static website hosting, backups, logs, etc.
- Data is stored in buckets and objects.

2. WHAT IS IAM?

- IAM manages access to AWS resources securely.
- Create Users, Groups and Policies.
- Attach policies to users or groups.
- Follow least privilege principle.

3. CREATE AN S3 BUCKET

```

resource "aws_s3_bucket" "my_bucket" {
  bucket          = "my-unique-bucket-12345"
  acl             = "private"
  force_destroy   = false
  tags = {
    Name      = "MyS3Bucket"
    Environment = "Dev"
  }
}
  
```

ARGUMENTS

Argument	Description
bucket	Unique name of the S3 bucket.
acl	Access control list for the bucket.
force_destroy	If true, bucket will be deleted even if not empty.
tags	Key-value pairs for resource identification.

4. ENABLE VERSIONING (OPTIONAL)

```

resource "aws_s3_bucket_versioning" "versioning_example" {
  bucket = aws_s3_bucket.my_bucket.id
  versioning_configuration {
    status = "Enabled"
  }
}
  
```

5. BLOCK PUBLIC ACCESS (BEST PRACTICE)

```

resource "aws_s3_bucket_public_access_block" "block_public" {
  bucket = aws_s3_bucket.my_bucket.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
  
```

6. CREATE AN IAM POLICY

```

resource "aws_iam_policy" "s3_read_policy" {
  name        = "s3-read-policy"
  description = "Policy to allow read access to S3"
  policy      = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [ "s3:ListBucket" ]
        Resource = [ "arn:aws:s3:::my-unique-bucket-12345" ]
      },
      {
        Effect = "Allow"
        Action = [ "s3:GetObject" ]
        Resource = [ "arn:aws:s3:::my-unique-bucket-12345/*" ]
      }
    ]
  })
}
  
```

7. CREATE AN IAM USER & ATTACH POLICY

```

resource "aws_iam_user" "dev_user" {
  name = "dev-user"
  tags = {
    Name = "DevUser"
  }
}

resource "aws_iam_user_policy_attachment" "attach_policy" {
  user       = aws_iam_user.dev_user.name
  policy_arn = aws_iam_policy.s3_read_policy.arn
}
  
```

10. SUMMARY

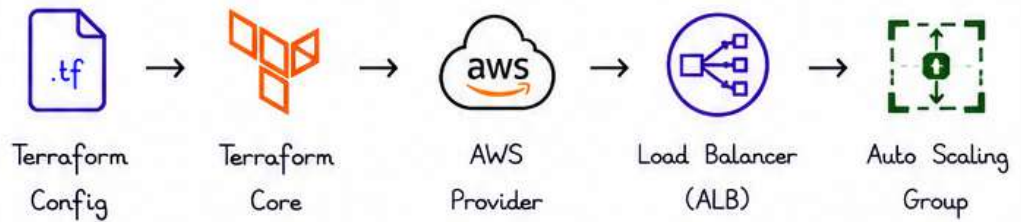
Service	Purpose
S3	Store and retrieve any amount of data.
Bucket	Container for objects in S3.
IAM	Manage users, groups and permissions.
Policy	Permissions defined in JSON.
User/Group	Identities to access AWS resources.

9. COMMON TERRAFORM COMMANDS

Command	Description
<code>terraform init</code>	Initialize the working directory.
<code>terraform plan</code>	Preview the changes.
<code>terraform apply</code>	Create or update resources.
<code>terraform destroy</code>	Destroy all managed resources.
<code>terraform output</code>	Show output values.

28. TERRAFORM — LOAD BALANCER & AUTO SCALING

AWS Load Balancer distributes traffic across multiple targets. Auto Scaling automatically adjusts the number of EC2 instances based on demand.



1. LOAD BALANCER (ALB)

```
resource "aws_lb" "app_alb" {
  name           = "app-alb"
  internal       = false
  load_balancer_type = "application"
  security_groups = ["sg-xxxxxxx"]
  subnets       = ["subnet-1111111", "subnet-2222222"]
}
```

Argument	Description
name	Name of the load balancer.
internal	false = Internet facing, true = Internal.
load_balancer_type	application (ALB) or network (NLB).
security_groups	Security group for the ALB.
subnets	Subnets where ALB will be created.

2. TARGET GROUP

```
resource "aws_lb_target_group" "app_tg" {
  name     = "app-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = "vpc-xxxxxxx"
  health_check {
    path            = "/"
    protocol        = "HTTP"
    interval        = 30
    healthy_threshold = 3
    unhealthy_threshold = 3
    timeout         = 5
  }
}
```

3. LISTENER

```
resource "aws_lb_listener" "app_listener" {
  load_balancer_arn = aws_lb.app_alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.app_tg.arn
  }
}
```

4. LAUNCH TEMPLATE

```
resource "aws_launch_template" "app_lt" {
  name_prefix = "app-lt-"
  image_id    = "ami-0c55b159cbfafa1f0"
  instance_type = "t3.micro"

  network_interfaces {
    associate_public_ip_address = true
    security_groups              = ["sg-xxxxxxx"]
  }
}
```

5. AUTO SCALING GROUP

```
resource "aws_autoscaling_group" "app_asg" {
  desired_capacity = 2
  min_size         = 1
  max_size         = 4
  vpc_zone_identifier = ["subnet-1111111", "subnet-2222222"]
  launch_template {
    id      = aws_launch_template.app_lt.id
    version = "$Latest"
  }
  target_group_arns = [aws_lb_target_group.app_tg.arn]
}
```

6. SCALING POLICY (CPU BASED)

```
resource "aws_autoscaling_policy" "cpu_policy" {
  name = "cpu-scaling-policy"
  autoscaling_group_name = aws_autoscaling_group.app_asg.name
  policy_type             = "TargetTrackingScaling"
  target_tracking_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
    }
    target_value = 60.0 # Keep average CPU around 60%
  }
}
```

Argument	Description
policy_type	TargetTrackingScaling for automatic scaling.
predefined_metric_type	Metric like ASGAverageCPUUtilization.
target_value	Desired average metric value.

7. FLOW

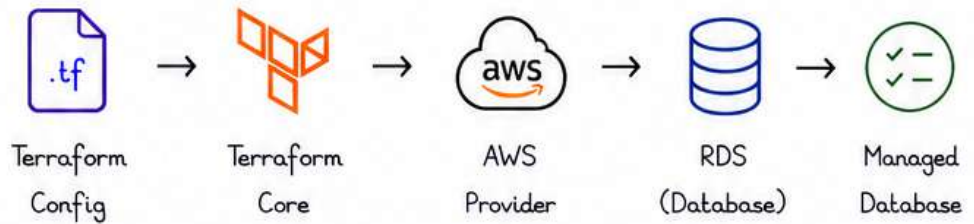
1. User requests → goes to ALB.
2. ALB forwards to healthy instances in Target Group.
3. If load increases → ASG adds instances.
4. If load decreases → ASG removes instances.

8. COMMON COMMANDS

- terraform init → Initialize configuration.
- terraform plan → Preview changes.
- terraform apply → Create resources.
- terraform destroy → Destroy all resources.
- terraform output → Show output values.

29. TERRAFORM — RDS

Amazon RDS (Relational Database Service) makes it easy to set up, operate, and scale a relational database in the cloud.



1. WHAT IS RDS?

- Managed relational database service by AWS.
- Supports MySQL, PostgreSQL, MariaDB, Oracle, SQL Server.
- Automated backups, patching, and high availability.
- Easy to scale and secure.

2. KEY FEATURES

- Multi-AZ for high availability.
- Automated backups and snapshots.
- Security groups and encryption.
- Read replicas for scaling read operations.

3. CREATE RDS INSTANCE

```

resource "aws_db_instance" "app_db" {
  identifier      = "app-db"
  engine          = "mysql"
  engine_version = "8.0"
  instance_class  = "db.t3.micro"
  allocated_storage = 20
  db_name         = "appdb"
  username        = "admin"
  password        = "StrongPass123!"
  skip_final_snapshot = true
  backup_retention_period = 7
  multi_az        = false
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  publicly_accessible = false
  storage_encrypted = true
}
  
```

Argument	Description
identifier	Unique name for the RDS instance.
engine	Database engine (mysql, postgres, etc.)
engine_version	Version of the database engine.
instance_class	Compute & memory capacity.
allocated_storage	Storage size in GB.
db_name	Initial database name.
username / password	Master username and password.
multi_az	Enable Multi-AZ for high availability.
backup_retention_period	Number of days to retain backups.
storage_encrypted	Enable storage encryption.

4. SECURITY GROUP

```

resource "aws_security_group" "rds_sg" {
  name        = "rds-sg"
  description = "Allow MySQL access"
  vpc_id      = "vpc-xxxxxx"

  ingress {
    from_port = 3306
    to_port   = 3306
    protocol  = "tcp"
    cidr_blocks = ["10.0.0.0/16"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
  
```

5. DB SUBNET GROUP

```

resource "aws_db_subnet_group" "rds_subnet" {
  name        = "rds-subnet-group"
  subnet_ids = [
    "subnet-11111111",
    "subnet-22222222"
  ]
  description = "Subnets for RDS"
}
  
```

6. PARAMETER GROUP (OPTIONAL)

```

resource "aws_db_parameter_group" "mysql_pg" {
  name        = "mysql-8-params"
  family      = "mysql8.0"

  parameter {
    name = "max_connections"
    value = "200"
  }
}
  
```

7. OUTPUTS

```

output "rds_endpoint" {
  description = "RDS endpoint"
  value       = aws_db_instance.app_db.endpoint
}

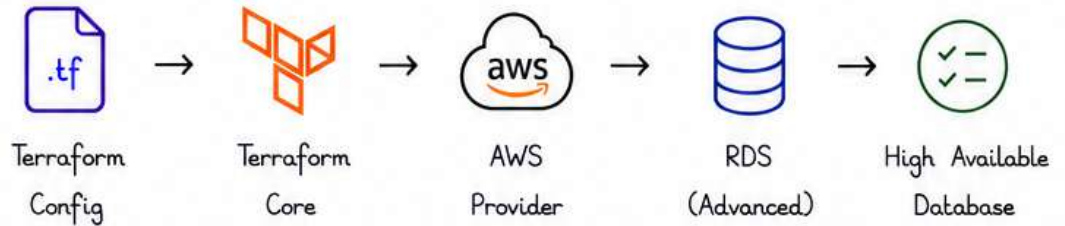
output "rds_port" {
  description = "RDS port"
  value       = aws_db_instance.app_db.port
}
  
```

8. COMMON COMMANDS

- | | |
|-------------------|----------------------------|
| terraform init | → Initialize configuration |
| terraform plan | → Preview changes |
| terraform apply | → Create RDS instance |
| terraform destroy | → Destroy RDS instance |
| terraform output | → Show output values |

30. TERRAFORM — RDS (Advanced)

This page covers RDS Subnet Group, Option Group, Read Replica and important settings.



1. RDS SUBNET GROUP

- Subnet group tells RDS the subnets (in different AZs) where DB instances can be created.
- Improves high availability.

```
resource "aws_db_subnet_group" "rds_subnet_group" {
  name          = "rds-subnet-group"
  subnet_ids    = ["subnet-11111111", "subnet-22222222"]
  description   = "Subnet group for RDS"
}
```

2. RDS OPTION GROUP

- Option group enables additional features like Oracle TDE, MySQL Audit Plugin, etc.

```
resource "aws_db_option_group" "rds_option_group" {
  name          = "rds-option-group"
  engine_name    = "mysql"
  major_engine_version = "8.0"
  option {
    option_name = "MARIADB_AUDIT_PLUGIN"
    option_settings {
      name = "SERVER_AUDIT_EVENTS"
      value = "CONNECT,QUERY"
    }
  }
}
```

3. CREATE RDS INSTANCE (WITH SUBNET GROUP & OPTION GROUP)

```
resource "aws_db_instance" "app_db" {
  identifier          = "app-db"
  engine              = "mysql"
  engine_version      = "8.0"
  instance_class      = "db.t3.micro"
  allocated_storage   = 20
  db_name             = "appdb"
  username            = "admin"
  password            = "StrongPass123!"
  db_subnet_group_name = aws_db_subnet_group.rds_subnet_group.name
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  option_group_name    = aws_db_option_group.rds_option_group.name
  multi_az            = true
  storage_encrypted    = true
  backup_retention_period = 7
  skip_final_snapshot = true
}
```

Argument	Description
db_subnet_group_name	Subnet group for RDS.
option_group_name	Option group for extra features.
multi_az	Creates standby in another AZ.
storage_encrypted	Encrypts storage.
backup_retention_period	Days to retain backups.
skip_final_snapshot	Skip snapshot on delete.

4. READ REPLICA (FOR SCALING READS)

```
resource "aws_db_instance" "app_db_read_replica" {
  identifier          = "app-db-replica"
  replicate_source_db = aws_db_instance.app_db.identifier
  instance_class      = "db.t3.micro"
  publicly_accessible = false
  skip_final_snapshot = true
}
```

Read Replica improves read performance and provides high availability.

5. COMMON SETTINGS (IMPORTANT)

- Multi-AZ** → High availability (standby in another AZ).
- Backup Retention** → Keep automatic backups for recovery.
- Storage Encrypted** → Encrypts data at rest.
- Security Group** → Allow only required DB ports.
- Subnet Group** → Use multiple AZ subnets.

6. OUTPUTS

```
output "rds_endpoint" {
  description = "RDS endpoint"
  value       = aws_db_instance.app_db.endpoint
}

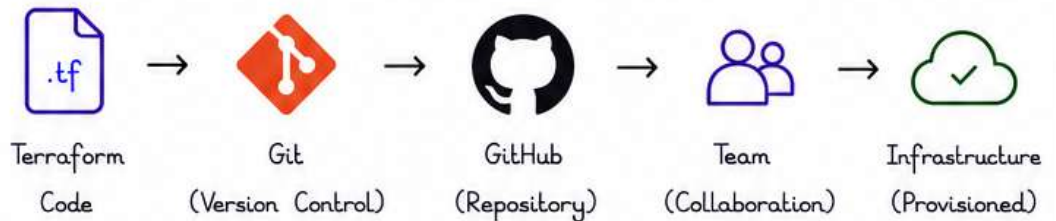
output "rds_port" {
  description = "RDS port"
  value       = aws_db_instance.app_db.port
}
```

7. COMMON COMMANDS

- terraform init** → Initialize configuration
- terraform plan** → Preview changes
- terraform apply** → Create or update RDS
- terraform destroy** → Destroy RDS resources
- terraform output** → Show output values

31. TERRAFORM WITH GIT & GITHUB

Using Git & GitHub with Terraform helps in version control, collaboration and tracking infrastructure changes.



1. WHY USE GIT & GITHUB WITH TERRAFORM?

- Version control for infrastructure code.
- Track changes and maintain history.
- Collaboration with team members.
- Rollback to previous working state.
- Store code securely in remote repository.

2. WORKFLOW OVERVIEW

1. Write Terraform code locally.
2. Initialize and test the code.
3. Add, Commit and Push to GitHub.
4. Team members Pull the latest code.
5. Apply changes to provision infrastructure.

```

terraform init
terraform plan
git add .
git commit -m "msg"
git push origin main
  
```

3. SETUP STEPS

Step	Action	Commands
1. Initialize Git	Initialize a Git repository in your project.	<code>git init</code>
2. Create .gitignore	Ignore unnecessary files.	<code>touch .gitignore</code>
3. Add Files	Add Terraform files to staging.	<code>git add .</code>
4. Commit	Commit changes with a message.	<code>git commit -m "Initial commit"</code>
5. Create GitHub Repo	Create a new repository on GitHub.	-
6. Push to GitHub	Link local repo with GitHub and push.	<code>git remote add origin <repo-url></code> <code>git push -u origin main</code>

4. SAMPLE .GITIGNORE

```

# Local terraform directory
.terraform/

# Terraform state files
*.tfstate
*.tfstate.*
crash.log

# Sensitive variables
*.tfvars
*.tfvars.json

# Override files
override.tf
override.tf.json
*_override.tf
*_override.tf.json

# .DS_Store (macOS)
.DS_Store
  
```

5. SAMPLE GIT COMMANDS

- Check status `git status`
- Add files `git add .`
- Commit changes `git commit -m "update"`
- Check branch `git branch`
- Create branch `git checkout -b feature-branch`
- Switch branch `git checkout main`
- Pull latest code `git pull origin main`
- Push changes `git push origin main`

6. BEST PRACTICES

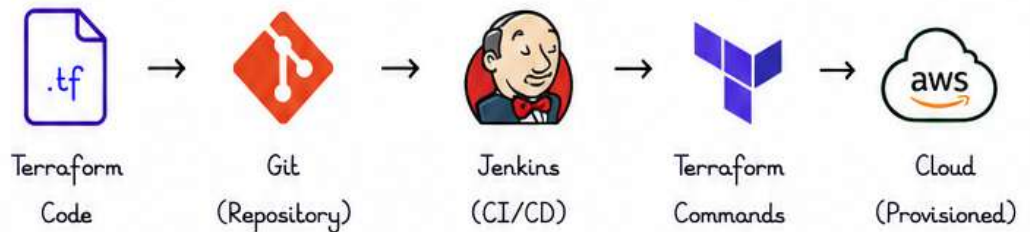
- Keep Terraform code in a dedicated repository.
- Use .gitignore to exclude state and sensitive files.
- Use branches for new features or changes.
- Review code before merging changes.
- Protect main branch with rules.

7. BENEFITS

- Collaboration and teamwork.
- Audit trail of all infrastructure changes.
- Easy rollback to previous versions.
- Secure and centralized code management.

32. TERRAFORM + JENKINS CI/CD

Jenkins automates Terraform workflow: Init, Plan, Apply and Destroy to provision infrastructure in a reliable and repeatable way.



1. WHY TERRAFORM + JENKINS?

- Automate infrastructure provisioning.
- Consistent and repeatable deployments.
- Version controlled infrastructure.
- Audit trail and logs for every run.
- Easy rollback by re-running previous version.

2. WORKFLOW

1. Developer pushes Terraform code to GitHub.
2. Jenkins job is triggered.
3. Jenkins pulls the code.
4. Terraform Init → Plan → Apply/Destroy.
5. Infrastructure is created/updated/deleted.



3. JENKINS PIPELINE EXAMPLE (Declarative)

```

pipeline {
  agent any
  environment {
    AWS_REGION = 'ap-south-1'
    TF_DIR = 'terraform'
  }
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/your-org/terraform-repo.git'
      }
    }
    stage('Terraform Init') {
      steps {
        sh 'cd $TF_DIR && terraform init -input=false'
      }
    }
    stage('Terraform Plan') {
      steps {
        sh 'cd $TF_DIR && terraform plan -out=tfplan'
      }
    }
    stage('Terraform Apply') {
      steps {
        sh 'cd $TF_DIR && terraform apply -auto-approve tfplan'
      }
    }
  }
  post {
    always {
      sh 'cd $TF_DIR && terraform destroy -auto-approve'
    }
  }
}
  
```

Stage	Purpose
Checkout	Pull Terraform code from Git.
Terraform Init	Initialize backend and plugins.
Terraform Plan	Show the execution plan.
Terraform Apply	Apply the changes.
Post (Destroy)	Cleanup resources (optional).

4. PREREQUISITES

- Jenkins installed and running.
- Terraform installed on Jenkins server.
- AWS credentials configured in Jenkins.
- GitHub repo with Terraform code.
- IAM user with required permissions.



5. JENKINS CREDENTIALS (AWS)

- Store AWS Access Key & Secret Key in Jenkins
- (Manage Jenkins → Credentials).
- Use in pipeline with withCredentials.

```

withCredentials([[[[Sclass: 'AmazonWebServicesCredentialsBinding',
  credentialsId: 'aws-creds']]]) {
  sh 'terraform init'
}
  
```



6. REMOTE BACKEND (S3 + DYNAMODB)

- Store Terraform state in S3.
- Use DynamoDB for state locking.

```

terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "envs/dev/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt      = true
  }
}
  
```



7. BEST PRACTICES

- Always use remote backend.
- Never hardcode secrets in code.
- Use parameterized builds for env (dev, qa, prod).
- Review Plan output before Apply.
- Use separate workspaces for environments.



8. COMMON TERRAFORM COMMANDS

- | | |
|--------------|--------------------------------------|
| • Initialize | terraform init |
| • Validate | terraform validate |
| • Plan | terraform plan -out=tfplan |
| • Apply | terraform apply -auto-approve tfplan |
| • Destroy | terraform destroy -auto-approve |
| • Format | terraform fmt -recursive |

33. TERRAFORM + GITHUB ACTIONS

GitHub Actions automates Terraform workflow directly from your GitHub repository. It helps in CI/CD of infrastructure as code.



1. WHY TERRAFORM + GITHUB ACTIONS?

- Automate Terraform workflows (CI/CD).
- No need to manage servers like Jenkins.
- Deep integration with GitHub.
- Secure secrets management.
- Free for public repositories.

2. WORKFLOW OVERVIEW

1. Developer pushes Terraform code to GitHub.
2. GitHub Actions workflow is triggered.
3. Terraform Init → Plan → Apply/Destory.
4. Infrastructure is created/updated/destroyed.



3. GITHUB ACTIONS WORKFLOW EXAMPLE

```

name: Terraform CI/CD
on:
  push:
    branches:
      - main
  workflow_dispatch:
jobs:
  terraform:
    runs-on: ubuntu-latest
    env:
      AWS_REGION: ap-south-1
      TF_DIR: infrastructure
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.6.6
      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ${ env.AWS_REGION }
      - name: Terraform Init
        run: terraform init -input=false
        working-directory: ${ env.TF_DIR }
      - name: Terraform Plan
        run: terraform plan -input=false -out=tfplan
        working-directory: ${ env.TF_DIR }
      - name: Terraform Apply
        if: github.ref == 'refs/heads/main'
        run: terraform apply -auto-approve tfplan
        working-directory: ${ env.TF_DIR }
  
```

Step	Purpose
Checkout Code	Pull code from GitHub repository.
Setup Terraform	Install specified Terraform version.
Configure AWS	Authenticate to AWS using secrets.
Terraform Init	Initialize backend and providers.
Terraform Plan	Generate and save the execution plan.
Terraform Apply	Apply the changes automatically.

4. GITHUB SECRETS (Settings → Secrets → Actions)

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- (Optional) TF_STATE_BUCKET, TF_STATE_KEY



5. TERRAFORM BACKEND (S3 + DYNAMODB)

```

terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "envs/dev/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "terraform-locks"
    encrypt       = true
  }
}
  
```



6. BEST PRACTICES

- Keep state file in remote backend.
- Use environment variables or secrets for credentials.
- Protect main branch with required reviews.
- Use separate workflows for plan and apply (if needed).
- Enable concurrency to avoid parallel runs.



7. BENEFITS

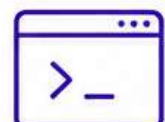
- Fully automated Infrastructure CI/CD.
- Version control and audit trail.
- Scalable and reliable.
- Easy rollback by re-applying previous version.
- No infrastructure to manage (serverless CI/CD).



8. COMMON TERRAFORM COMMANDS

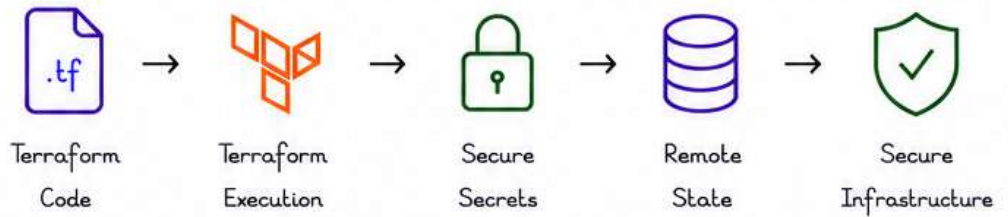
```

terraform init      → Initialize Terraform working directory.
terraform validate  → Validate Terraform configuration files.
terraform plan      → Create an execution plan.
terraform apply      → Apply the changes.
terraform destroy    → Destroy all resources.
terraform fmt        → Format Terraform configuration files.
  
```



34. TERRAFORM SECURITY & SECRETS BEST PRACTICES

Security is critical in Infrastructure as Code. Follow best practices to protect your code, state file, secrets and cloud resources.



1. SECURE YOUR STATE FILE

- Store state file in Remote backend (S3, Azure Blob, GCS).
- Enable encryption at rest and in transit.
- Enable versioning to recover from accidental changes.
- Restrict access using IAM policies.
- Enable state locking (DynamoDB / Blob lease / GCS lock).



2. MANAGE SECRETS SECURELY

- Never hardcode secrets in .tf files.
- Use environment variables for sensitive values.
- Use AWS Secrets Manager / SSM Parameter Store.
- Use Vault / Azure Key Vault / GCP Secret Manager.
- Pass secrets at runtime, not in code or state.



3. LIMIT ACCESS (LEAST PRIVILEGE)

- Use IAM users/roles with least privilege.
- Grant only required permissions.
- Use separate roles for CI/CD & Developers.
- Rotate access keys and secrets regularly.



4. PROTECT YOUR CODE

- Store code in private Git repositories.
- Enable branch protection and required reviews.
- Scan code for secrets (GitHub Advanced Security / TruffleHog).
- Do not commit .tfstate, .tfvars, or credentials.



5. SECURE PIPELINES & AUTOMATION

- Store secrets in CI/CD secret stores (GitHub Secrets, AWS Secrets Manager, etc.)
- Do not print secrets in logs.
- Use OIDC / IAM Roles for CI/CD (avoid static keys).
- Restrict pipeline permissions to minimum required.
- Mask sensitive values in pipeline logs.



6. ENCRYPTION

- Enable encryption for S3 buckets (SSE-S3 / SSE-KMS).
- Enable encryption for state file and snapshots.
- Use TLS for all API communications.



7. INPUT VALIDATION

- Use validation blocks in variables.
- Use terraform validate to check configuration.
- Use tfsec / checkov / terrascan for static code analysis.



8. TOOLS FOR SECURITY

Tool	Purpose
tfsec	Static analysis tool to find security issues in Terraform code.
checkov	Scans Terraform, CloudFormation, Kubernetes for misconfigurations.
terraform-compliance	Policy-as-Code with OPA for compliance checks.
tfLint	Terraform linter to enforce best practices.
GitHub Advanced Security	Secret scanning & code vulnerability scanning.

9. DO'S & DON'TS

DO'S ✓

- ✓ Use remote state with encryption & locking.
- ✓ Use IAM roles with least privilege.
- ✓ Rotate secrets regularly.
- ✓ Use environment variables or secret managers.
- ✓ Review & audit infrastructure changes.

DON'TS ✗

- ✗ Do not store secrets in code or state.
- ✗ Do not commit .terraform/ or .tfstate files.
- ✗ Do not use root credentials.
- ✗ Do not give * (admin) permissions.
- ✗ Do not expose state bucket publicly.

10. SAMPLE IAM POLICY (LEAST PRIVILEGE EXAMPLE)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3::my-terraform-state-bucket",
        "arn:aws:s3::my-terraform-state-bucket/*"
      ]
    }
  ]
}

```

Note :
Replace resources & actions as per your requirement and follow least privilege principle.



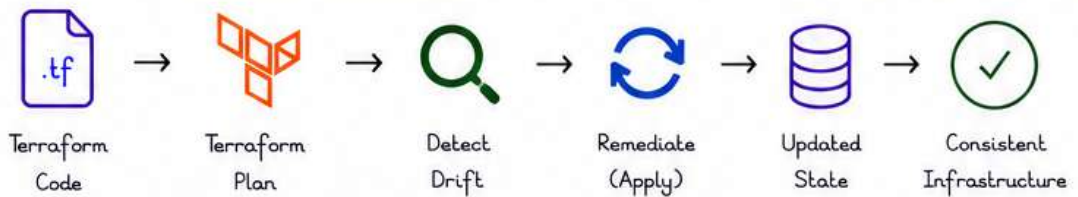
11. SUMMARY

- Protect state file with encryption, versioning and locking.
- Never store secrets in code or state.
- Use secret managers and environment variables.
- Apply least privilege for all users and pipelines.
- Scan code and enforce policies for security.
- Regularly audit and review your infrastructure.



35. TERRAFORM DRIFT DETECTION & REMEDIATION

Drift happens when real infrastructure changes differ from the Terraform state. Detect and remediate to keep infrastructure consistent.



1. WHAT IS DRIFT?

- Drift occurs when real infrastructure is manually changed outside Terraform.
- Terraform state no longer matches the real resources.
- It can cause unexpected behavior and failures.
- Examples: Manual change in EC2 instance type, security group rule, or deleted resource.



2. WHY DRIFT DETECTION IS IMPORTANT?

- Ensures infrastructure matches the desired state.
- Prevents configuration mismatches and outages.
- Improves security and compliance.
- Helps in troubleshooting and root cause analysis.
- Enables reliable automation and deployments.

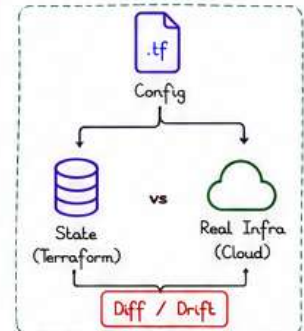


3. COMMANDS FOR DRIFT DETECTION

Command	Purpose
terraform plan	Shows changes between code, state and real infrastructure.
terraform refresh	Updates the state file with real infrastructure data.
terraform show	Displays current state file.
terraform state list	Lists all resources in state.
terraform state show <resource>	Shows details of a resource in state.

4. HOW DRIFT DETECTION WORKS?

1. terraform plan compares:
 - Terraform configuration
 - Current state file
 - Real infrastructure
2. If any difference is found, it shows the changes in the plan output.
3. You can review and decide to apply the changes or keep as is.



5. EXAMPLE: DRIFT DETECTION WITH terraform plan

```

$ terraform plan
~ aws_instance.web
  instance_type: "t2.micro" => "t3.micro" # drift detected
  tags.Name: "Web-Server" => "Web-Server-Updated" # drift detected
Plan: 0 to add, 1 to change, 0 to destroy.
  
```

Above output shows that real infrastructure is different from Terraform configuration and state.

6. DRIFT REMEDIATION

- Review the plan output carefully.
- If changes are expected, apply them:

```
terraform apply
```



- This will update the real infrastructure to match the configuration and state.
- If manual changes are correct, update the code and apply to sync.

7. METHODS TO PREVENT DRIFT

- Use Terraform for all infrastructure changes.
- Restrict manual changes in cloud console.
- Use IAM policies to limit direct access.
- Enable notifications and alerts for changes.
- Use drift detection in CI/CD pipelines.



8. TOOLS FOR DRIFT DETECTION

Tool	Purpose
Terraform Plan	Built-in drift detection.
Terraform Cloud	Drift detection & run tasks.
AWS Config	Track & detect resource changes.
Azure Policy	Audit & enforce compliance.
GCP Asset Inventory	Monitor & detect configuration drift.

9. AUTOMATE DRIFT DETECTION

You can automate drift detection in CI/CD pipeline.

```

- name: Terraform Plan (Drift Detection)
  run: |
    terraform init
    terraform plan -detailed-exitcode
- continue-on-error: true
env:
  TF_IN_AUTOMATION: "true"
  
```

Exit code 2 means drift detected.

0 = No changes
1 = Changes
2 = Drift detected

10. BEST PRACTICES

- Always run terraform plan before apply.
- Keep state file secure and up to date.
- Enforce all changes through Terraform.
- Review and approve changes in pipeline.
- Regularly run drift detection in production.
- Enable logging and monitoring for audit trail.



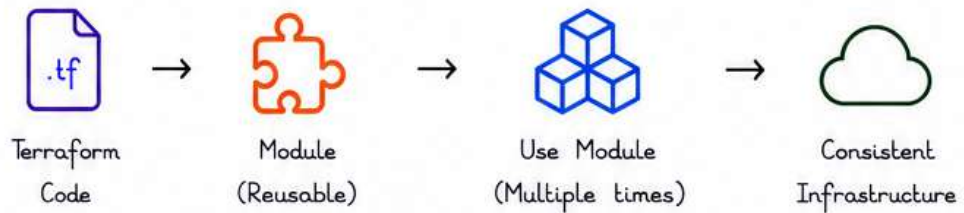
11. SUMMARY

- Drift occurs when real infrastructure differs from state.
- Use terraform plan to detect drift.
- Review and remediate using terraform apply.
- Prevent drift by restricting manual changes.
- Automate drift detection for reliable infrastructure.



36. TERRAFORM MODULES

Modules help to reusable, maintainable and scalable Terraform configurations.



1. WHAT IS A MODULE?

- A module is a container for multiple Terraform resources used together.
- It allows you to group, reuse and manage resources efficiently.
- Modules make code clean, DRY and maintainable.

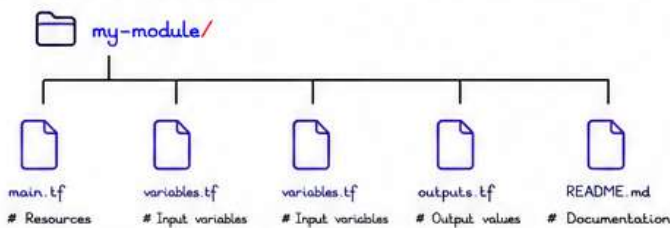


2. WHY USE MODULES?

- Reusability - Use the same code again and again.
- Maintainability - Easy to update in one place.
- Standardization - Enforce best practices.
- Scalability - Build complex infrastructure in simple way.



3. MODULE STRUCTURE



4. KEY CONCEPTS

- Input Variables - Pass values to module.
- Output Values - Return values from module.
- Source - Local path or remote (Registry, GitHub).
- Reusability - Call module multiple times.



5. EXAMPLE - CREATE A SIMPLE MODULE

main.tf

```
resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name
  acl    = "private"
  tags   = var.tags
}
```

variables.tf

```
variable "bucket_name" {
  type = string
}

variable "tags" {
  type = map(string)
  default = {}
}
```

outputs.tf

```
output "bucket_id" {
  value = aws_s3_bucket.this.id
}

output "bucket_arn" {
  value = aws_s3_bucket.this.arn
}
```

- main.tf → Resources definition
- variables.tf → Input variables
- outputs.tf → Output values
- README.md → Documentation



6. USING MODULE

```
module "s3_bucket" {
  source      = "../my-module" # local path
  bucket_name = "my-demo-bucket"
  tags = {
    Environment = "Dev"
    Project     = "Terraform"
  }
}

output "bucket_id" {
  value = module.s3_bucket.bucket_id
}

output "bucket_arn" {
  value = module.s3_bucket.bucket_arn
}
```

7. MODULE SOURCE TYPES

- Local Path

```
source = "../modules/vpc"
```
- Remote (GitHub)

```
source = "git::https://github.com/user/repo.git/vpc"
```
- Terraform Registry

```
source = "terraform-aws-modules/vpc/aws"
```
- Version (Optional)

```
version = "~> 4.0"
```



8. MODULE BEST PRACTICES

- Keep modules small and focused.
- Use meaningful variable names.
- Provide default values.
- Document inputs and outputs.
- Follow consistent naming.
- Version your modules.



9. ADVANTAGES

- ✓ Reduces code duplication
- ✓ Easy to reuse and share
- ✓ Simplifies large infrastructure
- ✓ Improves collaboration and consistency

10. DISADVANTAGES

- ✗ Overuse can make code complex
- ✗ Debugging can be tricky
- ✗ Learning curve for beginners

11. SUMMARY

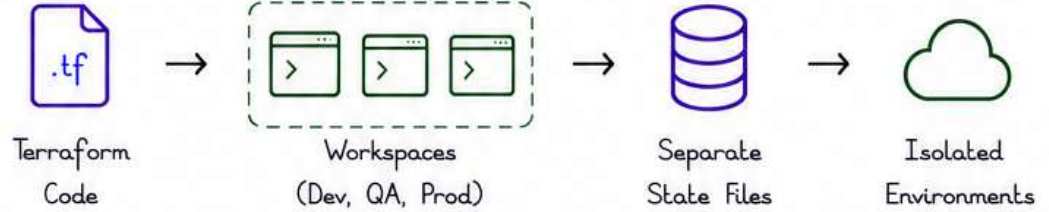
- Modules = Reusable Terraform code.
- Use variables for flexibility.
- Use outputs to pass values back.
- Modules make Terraform scalable and maintainable.



NOTE: Always test modules independently and version them for production use.

37. TERRAFORM WORKSPACES

Workspaces allow you to manage multiple environments with the same configuration.



1. WHAT IS A WORKSPACE?

- Workspaces are a way to manage multiple environments.
- Each workspace has its own state file.
- Same configuration, different state.
- Helps in isolation of environments.

2. WHY USE WORKSPACES?

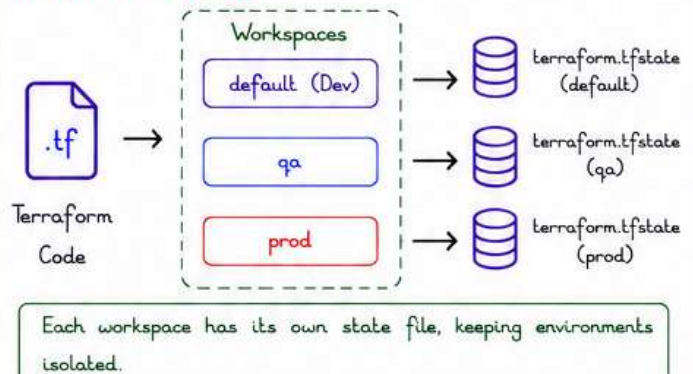
- Manage Dev, QA, Prod with same code.
- Keeps state files separate.
- Easy switching between environments.
- Avoids duplicating code.



3. WORKSPACE COMMANDS

Command	Purpose
<code>terraform workspace list</code>	List all workspaces.
<code>terraform workspace new <name></code>	Create a new workspace.
<code>terraform workspace select <name></code>	Switch to a workspace.
<code>terraform workspace show</code>	Show current workspace.
<code>terraform workspace delete <name></code>	Delete a workspace.

4. HOW IT WORKS?



5. EXAMPLE USAGE

- #List workspaces
`terraform workspace list`
- #Create a new workspace
`terraform workspace new qa`
- #Switch to qa workspace
`terraform workspace select qa`
- #Check current workspace
`terraform workspace show`
- #Delete workspace
`terraform workspace delete qa`

Example Output (list)

```
* default
  qa
  prod
```



6. BEST PRACTICES

- Use meaningful workspace names (dev, qa, prod).
- Keep backend same for all workspaces.
- Use variables for environment specific values.
- Protect prod workspace from accidental changes.
- Do not store secrets in code.



7. BACKEND & WORKSPACES

- Workspaces work best with remote backend (S3, etc.).
- State file of each workspace is stored separately.
- Example with S3 backend:

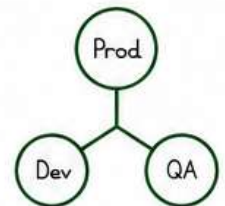
```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "global/s3/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "terraform-locks"
  }
}
```



S3
Backend

8. COMMON USE CASES

- Dev / QA / Prod environments.
- Separate environments for teams.
- Feature / Testing environments.
- Customer specific environments.



9. SUMMARY



- ✓ Workspaces help manage multiple environments with same code.
- ✓ Each workspace has its own state file.
- ✓ Use workspace commands to create, switch and manage.
- ✓ Keep environments isolated and follow best practices.

38. TERRAFORM INTERVIEW QUESTIONS + REVISION

Revise important
Terraform concepts
and interview questions
with answers.



Terraform
Code



Understand
Concepts



Interview
Questions



Revision &
Practice



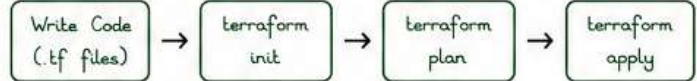
Be
Confident

1. WHAT IS TERRAFORM?

- Terraform is an Infrastructure as Code (IaC) tool by HashiCorp.
- It allows you to define, provision and manage infrastructure using declarative configuration.
- Supports multiple cloud providers.
- Uses a state file to track infrastructure.



2. BASIC WORKFLOW



- init** → Initialize the working directory.
plan → Preview the changes.
apply → Apply the changes to create/update resources.
destroy → Destroy the infrastructure.

3. KEY CONCEPTS

- Provider** - Plugin to interact with APIs (AWS, Azure, GCP).
- Resource** - The infrastructure object (EC2, S3, VPC).
- Data Source** - Fetch information about existing resources.
- Variables** - Reusable values (input variables).
- Outputs** - Return values after execution.
- State File** - Tracks real infrastructure (terraform.tfstate).
- Modules** - Reusable code for resources.
- Backend** - Where state file is stored (S3, GCS, etc).



4. INTERVIEW QUESTIONS & ANSWERS (BASICS)

- Q. What is Terraform?
Ans: Terraform is an IaC tool that helps to build, change and version infrastructure safely and efficiently.
- Q. What language does Terraform use?
Ans: HCL (HashiCorp Configuration Language).
- Q. What is a provider in Terraform?
Ans: Provider is a plugin that allows Terraform to interact with cloud or service APIs.
- Q. What is the purpose of terraform init?
Ans: It initializes the working directory, downloads providers and sets up the backend.
- Q. What is the purpose of terraform plan?
Ans: It shows an execution plan, what changes will be made without applying.



5. INTERVIEW QUESTIONS (INTERMEDIATE)

- Q. What is the difference between terraform plan and apply?
Ans: Plan shows the changes. Apply executes the changes.
- Q. What is a state file? Where is it stored?
Ans: State file keeps track of infrastructure. It can be stored in local, S3, GCS, Azure Blob, etc.
- Q. What are input variables in Terraform?
Ans: Variables allow dynamic values in configuration. Defined using variable blocks.
- Q. What is a module? Why use modules?
Ans: Module is a collection of resources. It promotes reuse, maintainability and consistency.
- Q. What is the difference between resource and data source?
Ans: Resource creates infrastructure. Data source reads existing infrastructure information.



6. INTERVIEW QUESTIONS (ADVANCED)

- Q. How does Terraform handle state locking?
Ans: State locking prevents multiple users from modifying the state file using DynamoDB (AWS) or GCS lock.
- Q. How can you manage secrets in Terraform?
Ans: Use environment variables, AWS Secrets Manager, HashiCorp Vault or SSM Parameter Store.
- Q. How do you manage multiple environments?
Ans: Using workspaces, separate state files, variables and backend configuration.
- Q. What is the purpose of depends_on in Terraform?
Ans: It creates explicit dependency between resources.
- Q. How can you import existing infrastructure in Terraform?
Ans: Use terraform import command.



7. IMPORTANT COMMANDS

Command	Purpose
terraform init	Initialize working directory
terraform plan	Show execution plan
terraform apply	Apply the changes
terraform destroy	Destroy the infrastructure
terraform fmt	Format configuration files
terraform validate	Validate configuration files
terraform show	Show current state
terraform output	Show output values
terraform import	Import existing infrastructure

8. BEST PRACTICES

- Use remote backend for state file.
- Enable state locking.
- Use modules for reusability.
- Use variables for flexibility.
- Do not hardcode secrets.
- Use version control (Git).
- Run terraform plan before apply.
- Follow naming conventions.



9. QUICK REVISION NOTES

- Terraform** → IaC Tool
- Language** → HCL
- Key Files** → *.tf, terraform.tfstate
- Workflow** → init → plan → apply
- State File** → Tracks real infra
- Modules** → Reuse code
- Providers** → Cloud connections
- Best Practice** → Remote state, locking, variables, modules, version control



10. SUMMARY



- ✓ Terraform helps to provision and manage infrastructure as code.
- ✓ Understand providers, resources, state, modules and backends.
- ✓ Practice commands and solve real-world scenarios.
- ✓ Follow best practices for secure and scalable infrastructure.

Revise Regularly
Practice More
Be Confident





:-: ABOUT AUTHOR :-:



Hi, I'm **Anush Agrawal**

- ★ DevOps Engineer / Cloud Enthusiast / Automation Lover
- ★ Passionate about IaC, CI/CD, DevOps culture and building reliable infrastructure.
- ★ I love sharing knowledge and helping DevOps aspirants to grow in their journey.
- ★ This Terraform Handbook is a result of my learning, real-world experience and continuous practice.

My Goal



To simplify Terraform for everyone and help you become job-ready with a practical approach.

Let's **Learn, Build, Automate**

and Grow Together !



:-: CONNECT WITH ME :-:



Instagram

@codewithanush

For DevOps Notes,
Tips, Reels & Updates



LinkedIn

linkedin.com/in/
anush-agrawal-26b557183

Let's Connect & Grow
Professionally



GitHub

github.com/anushagrawal123

Check out my Projects
& Contributions



Email

codewithanush@gmail.com

Feel free to reach out
for any queries !



★ -:-: THANK YOU :-: ★



Thank you for choosing this Terraform Handbook.
I hope this guide helps you on your **DevOps** journey
and brings value to your learning.



Keep Learning. Keep Building. Keep Automating.

You've Got This ! 💪



Keep this knowledge handy. Follow it step by step.
Stay **Consistent**, Stay **Curious** and Keep Shipping !



Scan to visit
my Instagram



DREAM BIG.

WORK HARD.

AUTOMATE EVERYTHING.

Plan • Code • Init • Plan • Apply • Monitor • Optimize • Repeat



Thank You for Reading Till the End! Happy Automating ! 🚀 ❤️